

HiDiTingV100 BT/BLE 软件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

HiDiTingV100 通过 API (Application Programming Interface) 向开发者提供接入和使用 Bluetooth 的相关接口, 包括 BT/BLE 的广播、扫描、连接等接口, 以及 A2DP、HFP、AVRCP 等 Profile 接口。本文档用于指导用户进行 BT/BLE 功能的开发。

目前版本提供 BT (经典蓝牙/低功耗蓝牙) 的发现、连接, 以及 GAP、GATT、HFP-HF、A2DP-SRC 等接口。

产品版本

与本文档对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用以下工程师:

- 技术支持工程
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 GAP 接口.....2

2.1 概述..... 2

2.2 开发流程..... 2

2.3 开发实例..... 18

2.3.1 经典蓝牙 GAP 开发实例..... 18

2.3.2 低功耗蓝牙 GAP 开发实例..... 19

2.4 注意事项..... 20

3 GATT server 接口22

3.1 概述..... 22

3.2 开发流程..... 22

3.3 开发实例..... 28

4 GATT client 接口.....31

4.1 概述..... 31

4.2 开发流程..... 31

4.3 开发实例..... 36

5 HFP-HF 接口.....38

5.1 概述..... 38

5.2 开发流程..... 38

5.3 开发实例..... 44

5.4 注意事项..... 45

6 A2DP-SRC 接口	46
6.1 概述	46
6.2 开发流程	46
6.3 开发实例	52
6.4 注意事项	53
7 A2DP-SNK 接口	54
7.1 概述	54
7.2 开发流程	54
7.3 开发实例	60
7.4 注意事项	61
8 SPP 接口	62
8.1 概述	62
8.2 开发流程	62
8.3 开发实例	67
8.4 注意事项	68
9 PBAP-PCE 接口	70
9.1 概述	70
9.2 开发流程	70
9.3 开发实例	77
9.4 注意事项	79
10 AVRCP-CT 接口	80
10.1 概述	80
10.2 开发流程	80
10.3 开发实例	87
11 AVRCP-TG 接口	89
11.1 概述	89
11.2 开发流程	89
11.3 开发实例	92
11.4 注意事项	93

12 PAN-PANU 接口	94
12.1 概述	94
12.2 开发流程	94
12.3 开发实例	98
13 HFP-AG 接口	100
13.1 概述	100
13.2 开发流程	100
13.3 开发实例	112
14 GATT 服务特性样例	114
14.1 概述	114
14.2 ANCS 客户端	114
14.3 AMS 客户端	120
15 常见问题定位指南	128
15.1 蓝牙连接相关问题	128
15.2 蓝牙广播相关问题	129
15.3 蓝牙配对相关问题	130
15.4 维测指令	131
15.5 BLE GATT 相关问题	131

表格目录

表 2-1 GAP 接口说明..... 3

表 2-2 gap_br_get_scan_mode 接口返回值说明 7

表 2-3 gap_get_pair_state 接口返回值说明..... 7

表 2-4 蓝牙协议栈状态..... 8

表 2-5 蓝牙 ACL 连接状态 8

表 2-6 蓝牙扫描状态 9

表 2-7 蓝牙绑定状态 9

表 2-8 蓝牙访问权限 9

表 2-9 蓝牙配对类型 9

表 2-10 蓝牙地址类型 10

表 2-11 蓝牙传输类型 10

表 2-12 其他函数返回值说明 10

表 2-13 BLE 广播过滤策略 11

表 2-14 BLE 广播类型..... 11

表 2-15 BLE 扫描类型..... 12

表 2-16 BLE 扫描过滤策略 12

表 2-17 BLE 扫描结果广播类型 13

表 2-18 BLE 扫描结果数据完整性 14

表 2-19 BLE PHY 类型 14

表 2-20 BLE 显示类型部分举例 14

表 2-21 蓝牙配对状态	15
表 2-22 ACL 链路状态.....	15
表 2-23 BLE 广播状态.....	15
表 2-24 蓝牙断链原因	16
表 2-25 设备输入输出能力	16
表 2-26 安全模式	16
表 2-27 执行结果状态码.....	17
表 3-1 GATT server 接口说明.....	22
表 3-2 GATT 特征属性定义.....	24
表 3-3 GATT 特征的操作权限	25
表 3-4 GATT 服务是否是首要服务	26
表 3-5 GATT 操作错误码	26
表 3-6 其他函数返回值说明	27
表 4-1 GATT client 接口说明.....	31
表 4-2 GATT 特征属性定义.....	33
表 4-3 GATT 操作错误码	33
表 4-4 其他函数返回值说明	35
表 5-1 HFP-HF 功能接口说明.....	39
表 5-2 hfp_hf_get_device_connect_state 接口返回值说明	40
表 5-3 其他函数返回值说明	41
表 5-4 接受一路呼叫时对其他呼叫的操作策略	42
表 5-5 电话呼叫状态类型	42
表 5-6 HFP 的音量调整类型	43
表 5-7 codec 编码信息.....	43
表 5-8 hfp_hf_get_sco_connect_state 接口返回值说明	43
表 6-1 A2DP Source 接口说明.....	47
表 6-2 a2dp_src_get_device_connect_state 接口返回值说明	48

表 6-3 a2dp_src_get_playing_state 接口返回值说明	48
表 6-4 a2dp_src_get_active_device 接口返回值说明	48
表 6-5 A2DP 编码格式类型	49
表 6-6 A2DP 配置编码格式支持状态	49
表 6-7 蓝牙通道参数类型定义	50
表 6-8 蓝牙音频通道事件类型定义	50
表 6-9 音频端口类型定义	51
表 6-10 其他函数返回值说明	52
表 7-1 A2DP Sink 接口说明	55
表 7-2 a2dp_snk_get_device_connect_state 接口返回值说明	56
表 7-3 a2dp_snk_get_playing_state 接口返回值说明	56
表 7-4 a2dp_snk_get_active_device 接口返回值说明	56
表 7-5 A2DP 编码格式类型	57
表 7-6 A2DP 配置编码格式支持状态	57
表 7-7 蓝牙通道参数类型定义	58
表 7-8 蓝牙音频通道事件类型定义	58
表 7-9 音频端口类型定义	59
表 7-10 其他函数返回值说明	59
表 8-1 SPP 接口说明	63
表 8-2 spp_server_create 接口返回值说明	64
表 8-3 spp_server_accept 接口返回值说明	64
表 8-4 spp_connect 接口返回值说明	65
表 8-5 is_spp_connected 接口返回值说明	65
表 8-6 spp_write 接口返回值说明	65
表 8-7 SPP socket 类型定义枚举	66
表 8-8 spp 连接状态说明	66
表 8-9 其他函数返回值说明	66

表 9-1 PBAP-PCE 功能接口说明	71
表 9-2 pbap_pce_get_device_connect_state 接口返回值说明	72
表 9-3 PBAP vCard 格式取值	72
表 9-4 PBAP 存储位置	73
表 9-5 PBAP 排序类型	73
表 9-6 PBAP 属性掩码取值	73
表 9-7 PBAP 错误码	76
表 9-8 其他函数返回值说明	76
表 10-1 AVRCP-CT 接口说明	81
表 10-2 avrcp_ct_get_device_connect_state 接口返回值说明	82
表 10-3 其他函数返回值说明	82
表 10-4 AVRCP 通知事件类型值说明	83
表 10-5 AVRCP 的按键操作	84
表 10-6 AVRCP 元素属性 ID	85
表 10-7 绝对音量范围（有效的取值范围[0x00, 0x7F]，表示 0% ~ 100%）	86
表 10-8 AVRCP 媒体播放状态	86
表 10-9 AVRCP 电量状态	87
表 10-10 AVRCP 系统状态	87
表 11-1 AVRCP-TG 接口说明	90
表 11-2 avrcp_tg_get_device_connect_state 接口返回值说明	91
表 11-3 其他函数返回值说明	91
表 12-1 PAN-PANU 接口说明	95
表 12-2 pan_service_net_state_get 接口返回值说明	96
表 12-3 其他函数返回值说明	96
表 12-4 发送网络数据包协议类型	97
表 12-5 发送数据包报文类型	97
表 13-1 HFP-AG 功能接口说明	101

表 13-2 HFP profile 连接状态	104
表 13-3 其他函数返回值说明	104
表 13-4 HFP 音频连接状态	105
表 13-5 HFP 的音量调整类型	105
表 13-6 codec 编码信息	105
表 13-7 电话呼叫状态类型	106
表 13-8 服务指示器的值	106
表 13-9 呼叫指示器的值	106
表 13-10 呼叫建立指示器的值	106
表 13-11 呼叫保持指示器的值	107
表 13-12 漫游指示器的值	107
表 13-13 网络事件类型	107
表 13-14 HFP 通话类型	108
表 13-15 AG 挂断类型	108
表 13-16 扩展的音频网关错误码	109
表 13-17 是否最后一个	110
表 13-18 CLCC 模式	110
表 13-19 是否多方通话	111
表 13-20 通话方向	111
表 13-21 号码信息服务值	111
表 13-22 运营商信息格式	111
表 13-23 运营商信息模式	112
表 14-1 ANCS 客户端接口说明	116
表 14-2 事件 ID	117
表 14-3 事件标记	117
表 14-4 命令 ID	118
表 14-5 分类 ID	118

表 14-6 通知属性 ID	119
表 14-7 应用属性 ID	119
表 14-8 活动 ID	119
表 14-9 订阅类型	120
表 14-10 ANCS 客户端写特征响应错误码	120
表 14-11 通知开关	120
表 14-12 AMS 客户端接口说明	122
表 14-13 远端命令 ID	123
表 14-14 实体 ID	124
表 14-15 播放器属性 ID	124
表 14-16 队列属性 ID	124
表 14-17 曲目属性 ID	125
表 14-18 随机模式	125
表 14-19 重复模式	125
表 14-20 AMS 客户端写和读特征响应错误码	126
表 14-21 实体更新标记	126
表 14-22 订阅类型	126
表 14-23 写特征类型	126
表 14-24 通知开关	127
表 15-1 初始化和连接相关回调函数	129
表 15-2 地址参数字节序	129
表 15-3 配对结果回调函数	130

1 概述

BrandyK100 通过 API (Application Programming Interface) 面向开发者提供 BT/BLE 功能的开发和应用接口, 包括 GAP、HFP-HF、A2DP-SRC 等接口。

各组件功能说明如下:

- GAP: Generic Access Profile, 包含蓝牙本地设置和经典蓝牙的发现和连接接口。
- HFP-HF: Hands-Free Profile 的 Hands-Free unit 角色接口, 包含控制手机进行蓝牙语音通话的功能相关接口。
- A2DP-SRC: Advanced Audio Distribution Profile 的 Source 角色接口, 包含音频传输相关接口。

说明

该文档描述各个模块功能的基本流程和 API 接口描述。

2 GAP 接口

- 2.1 概述
- 2.2 开发流程
- 2.3 开发实例
- 2.4 注意事项

2.1 概述

GAP 实现蓝牙设备开关控制、蓝牙设备信息管理、蓝牙设备的发现与被发现、连接与被连接、主动回连以及断开连接等功能。

2.2 开发流程

使用场景

打开蓝牙设备开关是使用蓝牙功能的首要条件，蓝牙启动后可进行设备信息管理，包括获取与设置本地设备名称、获取本地设备地址、获取配对信息、获取远端设备名称/设备类型/接收信号强度等。

通过设置蓝牙设备的发现模式，可使本地设备被其他设备发现、配对和连接；通过扫描，可发现其他设备，进一步与目标设备建立连接。当不再需要与远端设备连接时，可断开连接。

功能

GAP 功能提供的接口如表 2-1 所示。

表2-1 GAP 接口说明

接口名称	说明
enable_bt_stack	打开 BT 协议。
disable_bt_stack	关闭 BT 协议。
bluetooth_set_local_addr	设置本地设备地址。
bluetooth_get_local_addr	获取本地设备地址。
bluetooth_get_local_name	获取本地设备名称。
bluetooth_set_local_name	设置本地设备名称。
bluetooth_factory_reset	恢复蓝牙出厂设置。
gap_br_get_scan_mode	获取广播模式。
gap_br_set_bt_scan_mode	设置广播模式。
gap_get_paired_devices_num	获取配对设备数量。
gap_get_paired_devices_list	获取配对设备列表。
gap_get_pair_state	获取配对状态。
gap_remove_pair	删除配对设备。
gap_remove_all_pairs	删除所有配对设备。
bt_is_acl_connected	检查 ACL 链路是否连接。
gap_disconnect_remote_device	断开远端设备连接。
gap_connect_remote_device	与设备建立 ACL 连接。
gap_br_set_inquiry_paramters	设置经典蓝牙扫描参数。
gap_br_start_discovery	启动扫描。
gap_br_cancel_discovery	停止扫描。

接口名称	说明
is_bt_discovering	检查是否正在扫描。
gap_read_remote_rssi_value	获取远端设备 RSSI。
gap_get_device_name	获取远端设备名称。
gap_get_device_class	获取远端设备类型。只有存在远端设备信息时，才能获取到远端设备类型。没有远端设备信息时，获取远端设备类型为 0。
gap_reconnect_media_device	回连指定的已配对设备。
gap_register_callbacks	注册经典蓝牙 GAP 回调。
enable_ble	使能 BLE。
disable_ble	去使能 BLE。
gap_ble_connect_remote_device	与 BLE 设备建立 ACL 连接。
gap_ble_disconnect_remote_device	断开 BLE 设备连接。
gap_ble_connect_param_update	更新 BLE 连接参数。
gap_ble_remove_all_pairs	删除所有配对 BLE 设备。
gap_ble_remove_pair	根据蓝牙地址删除配对 BLE 设备。
gap_ble_get_pair_state	获取 BLE 设备配对状态。
gap_ble_get_paired_devices	获取已配对 BLE 设备列表。
gap_ble_get_paired_devices_num	获取已配对 BLE 设备数量。
gap_ble_pair_remote_device	与对端 BLE 设备进行配对。
gap_ble_set_adv_data	设置 BLE 广播数据。
gap_ble_set_adv_param	设置 BLE 广播参数。
gap_ble_start_adv	启动 BLE 广播。
gap_ble_stop_adv	停止 BLE 广播。
gap_ble_set_scan_parameters	设置扫描参数。

接口名称	说明
gap_ble_start_scan	启动 BLE 扫描。
gap_ble_stop_scan	停止 BLE 扫描。
gap_ble_set_local_name	配置 BLE 本地设备名称。
gap_ble_get_local_name	获取 BLE 本地设备名称。
gap_ble_set_local_appearance	配置本地设备外观。
gap_ble_get_local_addr	获取本地 BLE 设备蓝牙地址。
gap_ble_set_local_addr	配置本地 BLE 设备蓝牙地址。
gap_ble_register_callbacks	注册 BLE GAP 回调。
gap_ble_read_remote_device_rssi	读对端 BLE 设备 RSSI 值。
gap_ble_set_sec_param	设置安全参数。
gap_ble_get_connection_param	获取 BLE 连接参数。
bts_set_features	设置蓝牙特性。
bts_get_features	获取蓝牙特性配置。
gap_ble_set_ctkd_enable	配置一键双连开关。（仅支持 ios13 系统以上，安卓手机有兼容性问题，建议关闭）
gap_ble_get_bonded_devices	获取本端设备的已绑定 BLE 设备。
gap_ble_remove_all_bond_devices	删除所有 BLE 绑定设备信息。
bts_set_service_task_priority	配置 BT_SERVICE_TASK 任务优先级（请在调用【enable_bt_stack】函数之前调用）。
gap_bt_set_flow_limit	配置流控限制字节数。（请在调用【enable_bt_stack】函数之后调用，值范围是 3072 ~ 10240 Byte，协议栈默认限制是 10240Byte，如果数传时出现 B 核内存不足重启现象，可以适当调低该流控限制字节数，建议按照 1024Byte 递减。）

开发流程

GAP 开发的典型流程:

Acceptor:

- 步骤 1 调用 enable_bt_stack, 打开蓝牙开关。
- 步骤 2 调用 gap_register_callbacks, 注册 GAP 回调。
- 步骤 3 调用 bluetooth_set_local_name, 设置本地设备名称。
- 步骤 4 调用 gap_br_set_bt_scan_mode, 设置设备的可被发现模式。
- 步骤 5 手机扫描并连接蓝牙, 在回调函数 gap_acl_state_changed_callback 中实现连接结果处理。

----结束

Initiator:

- 步骤 1 调用 enable_bt_stack, 打开蓝牙开关。
- 步骤 2 调用 gap_register_callbacks, 注册 GAP 回调。
- 步骤 3 调用 bluetooth_set_local_name, 设置本地设备名称。
- 步骤 4 调用 gap_br_set_bt_scan_mode, 设置设备的可被发现模式。
- 步骤 5 调用 gap_br_start_discovery, 启动扫描。
- 步骤 6 调用 gap_connect_remote_device, 连接到目标设备。
- 步骤 7 在回调函数 gap_acl_state_changed_callback 中实现连接结果处理。

----结束

双模设备与苹果手机一键双连适配流程:

说明

一键双连仅限双模设备与苹果系统 ios13 以上使用的功能, 与安卓手机连接, 建议关闭一键双连功能。

- 蓝牙使能前, 调用 bts_set_features 接口, 配置自动回连特性。
- 蓝牙使能后, 建立 BLE 连接配对之前, 请执行以下步骤:

- 步骤 1 调用 gap_ble_set_sec_param 接口，打开 BLE 安全连接。
- 步骤 2 调用 gap_ble_set_ctkd_enable 接口，打开一键双连开关。该开关务必在 BLE 配对之前配置完成。
- 步骤 3 开启 BLE 广播，苹果手机 app 发起 BLE 连接配对，如果手表已开启自动回连，此时手表会自动回连 BT，成功后，手机系统蓝牙界面，只显示一个手表设备（BT 和 BLE 合成一个设备）；如果手表未开启自动回连，请执行步骤 4。
- 步骤 4 上层收到 BT 已配对状态上报时，调用 gap_reconnect_media_device 接口，地址入参用 BT 已配对状态上报的蓝牙地址，发起 BT 回连，成功后，手机系统蓝牙界面，只显示一个手表设备（BT 和 BLE 合成一个设备）。

----结束

返回值

获取广播模式返回值如表 2-2 所示。

表2-2 gap_br_get_scan_mode 接口返回值说明

序号	定义	实际数值	说明
1	GAP_SCAN_MODE_NONE	0	不可被发现模式。
2	GAP_SCAN_MODE_CONNECTABLE	1	可连接模式。
3	GAP_SCAN_MODE_GENERAL_DISCOVERABLE	2	普通发现模式。
4	GAP_SCAN_MODE_LIMITED_DISCOVERABLE	3	限制发现模式。
5	GAP_SCAN_MODE_CONNECTABLE_GENERAL_DISCOVERABLE	4	可连接且普通发现模式。
6	GAP_SCAN_MODE_CONNECTABLE_LIMITED_DISCOVERABLE	5	可连接且受限发现模式。

获取配对状态返回值如表 2-3 所示。

表2-3 gap_get_pair_state 接口返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	GAP_PAIR_NONE	1	未配对。
2	GAP_PAIR_PAIRING	2	配对中。
3	GAP_PAIR_PAIED	3	已配对。

获取设备类型 (Class of Device) 取值请参考：

<https://www.bluetooth.com/specifications/assigned-numbers/baseband/>

表2-4 蓝牙协议栈状态

序号	定义	实际数值	说明
1	BT_STACK_STATE_TURNING_ON	0	蓝牙协议栈正在初始化中。
2	BT_STACK_STATE_TURN_ON	1	蓝牙协议栈正在运行中。
3	BT_STACK_STATE_TURNING_OFF	2	蓝牙协议栈正在去初始化中。
4	BT_STACK_STATE_TURN_OFF	3	蓝牙协议栈已关闭。

表2-5 蓝牙 ACL 连接状态

序号	定义	实际数值	说明
1	GAP_ACL_STATE_CONNECTED	0	GAP ACL 已连接。
2	GAP_ACL_STATE_DISCONNECTED	1	GAP ACL 已断接。
3	GAP_ACL_STATE_LE_CONNECTED	2	BLE GAP ACL 已连接。
4	GAP_ACL_STATE_LE_DISCONNECTED	3	BLE GAP ACL 已

序号	定义	实际数值	说明
			断接。

表2-6 蓝牙扫描状态

序号	定义	实际数值	说明
1	GAP_DISCOVERY_STARTED	1	已开始扫描。
2	GAP_DISCOVERING	2	正在扫描。
3	GAP_DISCOVERY_STOPPED	3	扫描已停止。

表2-7 蓝牙绑定状态

序号	定义	实际数值	说明
1	GAP_BONDABLE_MODE_OFF	0	关闭蓝牙绑定模式。
2	GAP_BONDABLE_MODE_ON	1	打开蓝牙绑定模式。

表2-8 蓝牙访问权限

序号	定义	实际数值	说明
1	GAP_ACCESS_PERMISSION_UNKNOWN	0	未知。
2	GAP_ACCESS_PERMISSION_ALLOWED	1	允许访问。
3	GAP_ACCESS_PERMISSION_FORBIDDEN	2	禁止访问。

表2-9 蓝牙配对类型

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	GAP_PAIR_CONFIRM_TYPE_PIN_CODE	0x01	通过 pin code 配对。
2	GAP_PAIR_CONFIRM_TYPE_PASSKEY_DISPLAY	0x02	展示 PASSKEY。
3	GAP_PAIR_CONFIRM_TYPE_PASSKEY_INPUT	0x03	输入 PASSKEY。
4	GAP_PAIR_CONFIRM_TYPE_NUMERIC	0x04	配对数值。
5	GAP_PAIR_CONFIRM_TYPE_CONSENT	0x05	请求用户同意。

表2-10 蓝牙地址类型

序号	定义	实际数值	说明
1	BT_ADDRESS_TYPE_PUBLIC_DEVICE_ADDRESS	0x00	蓝牙公有地址。
2	BT_ADDRESS_TYPE_RANDOM_DEVICE_ADDRESS	0x01	蓝牙随机地址。

表2-11 蓝牙传输类型

序号	定义	实际数值	说明
1	BT_TRANSPORT_INVALID	0	未知传输类型。
2	BT_TRANSPORT_BR_EDR	1	传输类型为 BR/EDR。
3	BT_TRANSPORT_LE	2	传输类型为 BLE。

其他函数返回值如表 2-12 所示。

表2-12 其他函数返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	true	1	操作成功。
2	false	0	操作失败。

表2-13 BLE 广播过滤策略

序号	定义	实际数值	说明
1	GAP_BLE_ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY	0	处理所有设备的扫描和连接请求。
2	GAP_BLE_ADV_FILTER_ALLOW_SCAN_WLST_CON_ANY	1	处理所有连接请求，仅处理白名单的扫描请求。
3	GAP_BLE_ADV_FILTER_ALLOW_SCAN_ANY_CON_WLST	2	处理所有扫描请求，仅处理白名单的连接请求。
4	GAP_BLE_ADV_FILTER_ALLOW_SCAN_WLST_CON_WLST	3	仅处理白名单中扫描请求和连接请求。

表2-14 BLE 广播类型

序号	定义	实际数值	说明
1	GAP_BLE_ADV_CONN_SCAN_UNDIR	0	可连接可扫描非定向广播（默认）。
2	GAP_BLE_ADV_CONN_NONSCAN_DIR	1	可连接不可扫描定向广播。
3	GAP_BLE_ADV_NONCONN_SCAN_UNDIR	2	不可连接可扫描非定向广播。
4	GAP_BLE_ADV_NONCONN_NONSCAN_UNDIR	3	不可连接不可扫描非定向广播。

表2-15 BLE 扫描类型

序号	定义	实际数值	说明
1	GAP_BLE_SCAN_TYPE_PASSIVE	0	被动扫描。
2	GAP_BLE_SCAN_TYPE_ACTIVE	1	主动扫描。

表2-16 BLE 扫描过滤策略

序号	定义	实际数值	说明
1	GAP_BLE_SCAN_FILTER_POLICY_ACCEPT_ALL	0	接收所有广播，不接收目标地址不是本设备地址的定向广播。
2	GAP_BLE_SCAN_FILTER_POLICY_ONLY_WHITE_LIST	1	只接受白名单中设备的广播。
3	GAP_BLE_SCAN_FILTER_POLICY_ACCEPT_ALL_AND_RPA	2	接受所有的非定向广播、地址是可解析私有地址的广播方发送的定向广播以及发给该设备的定向广播。
4	GAP_BLE_SCAN_FILTER_POLICY_ONLY_WHITE_LIST_AND_RPA	3	接受白名单中的所有非定向广播、地址是可解析私有地址的广播方发送的定向广播以及发给该设备的定向广播。

表2-17 BLE 扫描结果广播类型

序号	定义	实际数值	说明
1	GAP_BLE_EVT_NON_CONNECTABLE_NON_SCANNABLE	0x00	扩展的不可连接不可扫描非定向广播。
2	GAP_BLE_EVT_NON_CONNECTABLE_NON_SCANNABLE_DIRECTED	0x04	扩展的不可连接不可扫描定向广播。
3	GAP_BLE_EVT_CONNECTABLE	0x01	扩展的可连接非定向广播。
4	GAP_BLE_EVT_CONNECTABLE_DIRECTED	0x05	扩展的可连接定向广播。
5	GAP_BLE_EVT_SCANNABLE	0x02	扩展的可扫描非定向广播。
6	GAP_BLE_EVT_SCANNABLE_DIRECTED	0x06	扩展的可扫描定向广播。
7	GAP_BLE_EVT_LEGACY_NON_CONNECTABLE	0x10	传统的不可连接非定向广播。
8	GAP_BLE_EVT_LEGACY_SCANNABLE	0x12	传统的可扫描非定向广播。
9	GAP_BLE_EVT_LEGACY_CONNECTABLE	0x13	传统的可连接可扫描非定向广播。
10	GAP_BLE_EVT_LEGACY_CONNECTABLE_DIRECTED	0x15	传统的可连接定向广播。
11	GAP_BLE_EVT_LEGACY_SCAN_RSP_TO_ADV_SCAN	0x1A	传统的与 ADV_SCAN_IND 对应的扫描响应广播。
12	GAP_BLE_EVT_LEGACY_SCAN_RSP_TO_ADV	0x1B	传统的与 ADV_IND 对应的扫描响应广播。

表2-18 BLE 扫描结果数据完整性

序号	定义	实际数值	说明
1	GAP_BLE_DATA_COMPLETE	0x00	完整数据或最后一个片段。
2	GAP_BLE_DATA_INCOMPLETE_MORE_TO_COME	0x01	不完整的数据。
3	GAP_BLE_DATA_INCOMPLETE_TRUNCATED	0x02	被截断不完整的数据。

表2-19 BLE PHY 类型

序号	定义	实际数值	说明
1	GAP_BLE_PHY_NO_PACKET	0x00	无广播包。
2	GAP_BLE_PHY_1M	0x01	1M PHY。
3	GAP_BLE_PHY_2M	0x02	2M PHY。
4	GAP_BLE_PHY_CODED	0x03	Coded PHY。

表2-20 BLE 显示类型部分举例

序号	定义	实际数值	说明
1	GAP_BLE_APPEARANCE_TYPE_UNKOWN	00	未知类型。
2	GAP_BLE_APPEARANCE_TYPE_GENERIC_PHONE	64	通用手机。
3	GAP_BLE_APPEARANCE_TYPE_GENERIC_COMPUTER	128	通用电脑。
4	GAP_BLE_APPEARANCE_TYPE_GENERIC_WATCH	192	通用手表。
5	GAP_BLE_APPEARANCE_TYPE_GENERIC_DISPLAY	320	通用显示器。
6	GAP_BLE_APPEARANCE_TYPE_GENERIC_HID	960	通用 HID。

序号	定义	实际数值	说明
	RIC_HID		
7	GAP_BLE_APPEARANCE_TYPE_KEYBOARD	961	键盘。
8	GAP_BLE_APPEARANCE_TYPE_MOUSE	962	鼠标。
9	GAP_BLE_APPEARANCE_TYPE_DIGITAL_PEN	967	电子笔。

表2-21 蓝牙配对状态

序号	定义	实际数值	说明
1	GAP_BLE_PAIR_NONE	0x01	未配对状态。
2	GAP_BLE_PAIR_PAIRING	0x02	正在配对。
3	GAP_BLE_PAIR_PAIRING	0x03	已完成配对。

表2-22 ACL 链路状态

序号	定义	实际数值	说明
1	GAP_BLE_STATE_DISCONNECTED	0	BLE GAP ACL 已断连。
2	GAP_BLE_STATE_CONNECTED	1	BLE GAP ACL 已连接。

表2-23 BLE 广播状态

序号	定义	实际数值	说明
1	ADV_STATUS_STOPPED	0x00	广播停止。
2	ADV_STATUS_ADVERTISING	0x01	正在广播。

表2-24 蓝牙断链原因

序号	定义	实际数值	说明
1	GAP_BLE_DISCONN_UNKNOWN	0x00	未知原因断链。
2	GAP_BLE_ERR_CONN_TIMEOUT	0x8	连接超时断链。
3	GAP_BLE_DICSCONNECT_BY_REMOTE_USER	0x13	远端用户断链。
4	GAP_BLE_CONN_TERMINATE_BY_LOCAL_HOST	0x16	本端 HOST 断链。

表2-25 设备输入输出能力

序号	定义	实际数值	说明
1	GAP_BLE_IO_CAPABILITY_DISPLAYONLY	0	只展示。
2	GAP_BLE_IO_CAPABILITY_DISPLAYYESNO	1	展示，并且可以选择 Yes 或者 No。
3	GAP_BLE_IO_CAPABILITY_KEYBOARDONLY	2	只支持键盘。
4	GAP_BLE_IO_CAPABILITY_NOINPUTNOOUTPUT	3	没有输入输出。
5	GAP_BLE_IO_CAPABILITY_KEYBOARDDISPLAY	4	支持键盘和展示。

表2-26 安全模式

序号	定义	实际数值	说明
1	GAP_BLE_GAP_SECURITY_MODE1_LEVEL1	0	没有安全能力。
2	GAP_BLE_GAP_SECURITY_MODE1_LEVEL2	1	不需要认证，基于链路进行配对和加密。
3	GAP_BLE_GAP_SECURITY_MODE1_LEVEL3	2	需要认证，基于链路进行配对和加

序号	定义	实际数值	说明
			密。
4	GAP_BLE_GAP_SECURITY_MODE1_LE VEL4	3	需要认证, 基于链路采用 ECDH 算法进行加密和配对。
5	GAP_BLE_GAP_SECURITY_MODE2_LE VEL1	4	不需要认证, 基于数据进行配对和加密。
6	GAP_BLE_GAP_SECURITY_MODE2_LE VEL2	5	需要认证, 基于数据进行配对和加密。

表2-27 执行结果状态码

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。

序号	定义	实际数值	说明
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

2.3 开发实例

GAP 模块基于经典蓝牙和低功率蓝牙分别给出 GAP 连接流程实例。

2.3.1 经典蓝牙 GAP 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_br_gap.h” 实现 BR 设备连接实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/bt_core” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 实现 BR 回调函数。

按照 gap_discovery_result_callback 回调函数为例：

```
static void sample_gap_discovery_result_callback(const bd_addr_t *bd_addr)
{
    bool ret = false;
    unsigned char length = BD_NAME_LEN;
    unsigned char *device_name = (unsigned char *) (void *) bt_os_new(length);
    if (device_name == NULL) {
        return ret;
    }
    (void)memset_s(device_name, length, 0, length);
    ret = gap_get_device_name(bd_addr, device_name, &length);
    int cod = gap_get_device_class(bd_addr);
    ret = gap_read_remote_rssi_value(bd_addr);
    return ret;
}
```

步骤 3 注册 BR 回调函数。

```
gap_call_backs_t cb = {
    .discovery_result_callback = sample_gap_discovery_result_callback,
    // 其他回调函数按需注册
};
int ret = gap_register_callbacks(&cb);
return ret;
```

步骤 4 启动扫描。

```
bool res = gap_br_start_discovery();
return (uint32_t)res;
```

步骤 5 连接设备。

```
errcode_t ret = gap_connect_remote_device(bd_addr);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

----结束

2.3.2 低功耗蓝牙 GAP 开发实例

以下基于 SDK 包 “include/middleware/services/bts/ble/bts_le_gap.h” 中的函数实现 BLE 设备广播实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/bt_core” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

- 双模设备可参考以下代码实现：

```
errcode_t ret = enable_bt_stack();
return ret;
```

- 单模 BLE 设备参考以下代码实现：

```
errcode_t ret = enable_ble();
if (ret != ERRCODE_BT_SUCCESS) {
    return BLE_AT_FAIL;
}
return BLE_AT_SUCC;
```

步骤 2 实现 BLE 回调函数。

按照 gap_ble_connect_state_changed_callback 回调函数为例：

```
static void central_demo_conn_state_change_cbk(
    uint16_t conn_id, bd_addr_t *addr, gap_ble_conn_state_t conn_state, gap_ble_pair_state_t
```

```
pair_state) {
    if (addr == NULL) {
        return;
    }
    if (conn_state == GAP_BLE_STATE_CONNECTED) {
        errcode_t ret;
        ret = gattc_exchange_mtu_req(g_gatt_client_id, conn_id, CENTRAL_DEMO_MTU);
        bt_uuid_t uuid = {0};
        ret = gattc_discovery_service(g_gatt_client_id, conn_id, &uuid);
    }
}
```

步骤 3 注册 BLE 回调函数。

```
errcode_t ret;
gap_ble_callbacks_t gap_ble_cb = {
    .conn_state_change_cb = central_demo_conn_state_change_cbk,
    // 其他回调函数按需注册
};
/* gap 回调注册 */
ret = gap_ble_register_callbacks(&gap_ble_cb);
if (ret != ERRCODE_BT_SUCCESS) {
    return ret;
}
```

步骤 4 发送 BLE 广播。

```
errcode_t ret = gap_ble_start_adv(adv_id);
return ret;
```

----结束

2.4 注意事项

- 对于穿戴设备 BrandyK100 上经典蓝牙连接最多只支持一路耳机连接和一路手机连接。
- 为使发现设备列表更简洁，扫描结果只上报音视频类设备（Major Device Class 为 Audio/Video）。
- 连接耳机/音箱时，若扫描不到设备，请先检查设备是否已在配对设备列表中，或者设备是否已与其他设备配对（此情况下需要先清除设备端配对信息）。

- `bts_set_features` 接口当前支持配置开启或者关闭经典蓝牙自动回连功能，限定只能在使能协议栈之前配置。
- `gap_reconnect_media_device` 接口支持回连已配对过的经典蓝牙设备，仅限在经典蓝牙自动回连功能关闭的场景下使用。
- `gap_ble_set_ctkd_enable` 接口配置一键双连开关，一键双连仅限与苹果手机的一键双连，且 ios13 以上系统，与安卓手机连接时，建议关闭一键双连功能。

Draft

3 GATT server 接口

- 3.1 概述
- 3.2 开发流程
- 3.3 开发实例

3.1 概述

GATT 是一个基于蓝牙 GAP 连接的发送和接收数据的通用规范，支持在两个蓝牙设备间进行数据传输。

3.2 开发流程

使用场景

GATT server 主要用来接收对端设备的命令和请求，给对端设备发送响应、指示或者通知。

功能

GATT server 功能提供的接口如表 3-1 所示。

表3-1 GATT server 接口说明

接口名称	说明
gatts_register_server	Gatt server 注册。根据传入的 UUID 注册

接口名称	说明
	server, 回调函数返回 server ID。
gatts_unregister_server	Gatt server 去注册。
gatts_add_service	添加 service, 结果以异步回调的方式返回。
gatts_add_characteristic	添加 characteristic 到指定的 service, 结果以异步回调的方式返回。
gatts_add_descriptor	添加 descriptor 到对应的 characteristic。包含当前 Characteristic 的描述信息、配置信息、表示格式信息等。结果以异步回调的方式返回。
gatts_add_service_sync	添加 service, 结果同步返回。
gatts_add_characteristic_sync	添加 characteristic 到指定的 service, 结果同步返回。
gatts_add_descriptor_sync	添加 descriptor 到对应的 characteristic。包含当前 Characteristic 的描述信息、配置信息、表示格式信息等。结果同步返回。
gatts_start_service	启动 service。
gatts_stop_service	停止 service。
gatts_delete_service	删除指定的 service。
gatts_delete_all_services	清除所有的 service。
gatts_notify_indicate	给远端 client 发送 indication/notification。
gatts_set_mtu_size	配置 MTU 大小。
gatts_register_callbacks	注册 Gatt server 回调函数。
gatts_notify_indicate_by_uuid	根据特征 UUID 发送通知或指示。
gatts_send_response	向对端发送响应。目前为自动发送。

开发流程

GATT server 开发的典型流程：添加服务和特征及描述信息并启动服务。

- 步骤 1 调用 enable_bt_stack，打开蓝牙开关。
- 步骤 2 调用 gatts_register_callbacks，注册 GATT server 回调。
- 步骤 3 调用 gatts_register_server，创建一个 server。
- 步骤 4 调用 gatts_add_service，根据 UUID 创建 service，结果以异步回调的方式返回，需要同步返回的，可以调用 gatts_add_service_sync。
- 步骤 5 调用 gatts_add_characteristic，对创建的服务添加特征值，结果以异步回调的方式返回，需要同步返回的，可以调用 gatts_add_characteristic_sync。
- 步骤 6 调用 gatts_add_descriptor，对服务中的特征添加描述信息，结果以异步回调的方式返回，需要同步返回的，可以调用 gatts_add_descriptor_sync。
- 步骤 7 调用 gatts_start_service，启动 service。
- 步骤 8 调用 gap_ble_set_adv_param，设置广播参数。
- 步骤 9 调用 gap_ble_start_adv，启动广播，等待对端连接。

----结束

表3-2 GATT 特征属性定义

序号	定义	实际数值	说明
1	GATT_CHARACTER_PROPERTY_BIT_BROADCAST	0x01	允许广播 characteristic value。
2	GATT_CHARACTER_PROPERTY_BIT_READ	0x02	允许读 characteristic value。
3	GATT_CHARACTER_PROPERTY_BIT_WRITE_NO_RSP	0x04	允许不响应写 characteristic value。
4	GATT_CHARACTER_PROPERTY_BIT_WRITE	0x08	允许写 characteristic value 并且需要给 client 回复。
5	GATT_CHARACTER_PROPERTY_BIT_NOTIFY	0x10	允许把 characteristic

序号	定义	实际数值	说明
			value 通知给 client, 并且不需要 client 回复确认。
6	GATT_CHARACTER_PROPERTY_BIT_INDICATE	0x20	允许把 characteristic value 指示给 client, 并且需要 client 回复确认。
7	GATT_CHARACTER_PROPERTY_BIT_SIGNED_WRITE	0x40	允许对 characteristic value 进行签名写入。
8	GATT_CHARACTER_PROPERTY_BIT_EXTENDED_PROPERTY	0x80	在特征扩展特性描述符中定义了附加的特征特性。

表3-3 GATT 特征的操作权限

序号	定义	实际数值	说明
1	GATT_ATTRIBUTE_PERMISSION_READ	0x01	可读。
2	GATT_ATTRIBUTE_PERMISSION_WRITE	0x02	可写。
3	GATT_ATTRIBUTE_PERMISSION_ENCRYPTION_NEED	0x04	需要加密。
4	GATT_ATTRIBUTE_PERMISSION_AUTHENTICATION_NEED	0x08	需要认证。
5	GATT_ATTRIBUTE_PERMISSION_AUTHORIZATION_NEED	0x10	需要授权。
6	GATT_ATTRIBUTE_PERMISSION_MITM_NEED	0x20	需要 MITM 保护。

表3-4 GATT 服务是否是首要服务

序号	定义	实际数值	说明
1	true	1	是。
2	false	0	否。

表3-5 GATT 操作错误码

序号	定义	实际数值	说明
1	GATT_STATUS_SUCCESS	0x00	执行成功错误码。
2	GATT_STATUS_INVALID_HANDLE	0x01	无效 handle 错误码。
3	GATT_STATUS_READ_NOT_PERMITTED	0x02	不可读错误码。
4	GATT_STATUS_WRITE_NOT_PERMITTED	0x03	不可写错误码。
5	GATT_STATUS_INVALID_PDU	0x04	无效 PDU 错误码。
6	GATT_STATUS_INSUFFICIENT_AUTHENTICATION	0x05	未认证的读写错误码。
7	GATT_STATUS_REQUEST_NOT_SUPPORTED	0x06	不支持 client 发送的请求。
8	GATT_STATUS_INVALID_OFFSET	0x07	无效偏移的读写错误码。
9	GATT_STATUS_INSUFFICIENT_AUTHORIZATION	0x08	未授权错误码。
10	GATT_STATUS_PREPARE_QUEUE_FULL	0x09	排队的 prepare writes 太多, 队列已满错误码。
11	GATT_STATUS_ATTRIBUTE_NOT_FOUND	0x0A	属性不存在错误码。
12	GATT_STATUS_ATTRIBUTE_NOT_LONG	0x0B	该 attribute 不是

序号	定义	实际数值	说明
	G		long attribute, 不能使用 ATT_READ_BLOCK_REQ PDU。
13	GATT_STATUS_INSUFFICIENT_ENCRYPTION_KEY_SIZE	0x0C	用于加密此连接的加密密钥大小不足。
14	GATT_STATUS_INVALID_ATTRIBUTE_VALUE_LENGTH	0x0D	无效的属性值长度。
15	GATT_STATUS_UNLIKELY_ERROR	0x0E	不可能的错误码。
16	GATT_STATUS_INSUFFICIENT_ENCRYPTION	0x0F	加密不足错误码。
17	GATT_STATUS_UNSUPPORTED_GROUP_TYPE	0x10	不支持的 grouping attribute 错误码。
18	GATT_STATUS_INSUFFICIENT_RESOURCES	0x11	资源不足错误码。
19	GATT_STATUS_DATABASE_OUT_OF_SYNC	0x12	服务端与客户端未同步数据库错误码。
20	GATT_STATUS_VALUE_NOT_ALLOWED	0x13	非法的参数值错误码。

表3-6 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。

序号	定义	实际数值	说明
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

3.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/ble” 中相关 GATT 头文件实现 GATT 服务添加实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/ble_hid_take_photo_server.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

- 双模设备可参考以下代码实现：

```
errcode_t ret = enable_bt_stack();
return ret;
```

- 单模 BLE 设备参考以下代码实现：

```
errcode_t ret = enable_ble();
if (ret != ERRCODE_BT_SUCCESS) {
    return BLE_AT_FAIL;
}
```



```
return BLE_AT_SUCC;
```

步骤 2 实现 GATT 回调函数。

按照 gatts_add_service_callback 回调函数为例：

```
static void ble_server_service_add_cbk(uint8_t server_id, bt_uuid_t *uuid, uint16_t handle,
errcode_t status) {
    if (status != ERRCODE_BT_SUCCESS) {
        return;
    }
    bt_uuid_t hid_service_uuid = {0};
    bts_data_to_uuid_len2(BLE_UUID_HUMAN_INTERFACE_DEVICE, &hid_service_uuid);
    if (bt_gatts_cmp_uuid(uuid, &hid_service_uuid) == true) {
        ble_hid_add_characters_and_descriptors(server_id, handle);
        gatts_start_service(server_id, handle);
    }
}
```

步骤 3 注册 GATT server 回调函数。

```
gatts_callbacks_t cb = {
    .add_service_cb = ble_server_service_add_cbk,
    // 其他回调函数按需注册
};
errcode_t ret = gatts_register_callbacks(&cb);
return ret;
```

步骤 4 注册 server。

```
bt_uuid_t app_uuid = {0};
errcode_t ret = gatts_register_server(&app_uuid, &g_server_id);
return ret;
```

步骤 5 添加服务。

```
static void ble_hid_add_service(void)
{
    bt_uuid_t hid_service_uuid = {0};
    bts_data_to_uuid_len2(BLE_UUID_HUMAN_INTERFACE_DEVICE, &hid_service_uuid);
    gatts_add_service(BLE_HID_SERVER_ID, &hid_service_uuid, true);
}
```

步骤 6 添加特征。

```
static void ble_hid_add_character_output_report(uint8_t server_id, uint16_t svc_handle)
{

```

```
/* HID output report value for test */
uint8_t output_report_value[] = {0x00};
gatts_add_chara_info_t character;
bts_data_to_uuid_len2(BLE_UUID_REPORT, &character.chara_uuid);
character.permissions = GATT_ATTRIBUTE_PERMISSION_READ;
character.properties = HID_OUTPUT_REPORT_PROPERTIES;
character.value_len = (uint16_t)sizeof(output_report_value);
character.value = output_report_value;
gatts_add_characteristic(server_id, svc_handle, &character);
ble_hid_add_descriptor_report_reference(server_id, svc_handle, false);
}
```

步骤 7 启动服务。

```
errcode_t ret = gatts_start_service(server_id, handle);
return ret;
```

----结束

4 GATT client 接口

4.1 概述

4.2 开发流程

4.3 开发实例

4.1 概述

GATT 是一个基于蓝牙 GAP 连接的发送和接收数据的通用规范，支持在两个蓝牙设备间进行数据传输。

4.2 开发流程

使用场景

GATT client 主要给对端发送命令和请求，接收对端回复的响应、指示和通知。

功能

GATT client 功能提供的接口如表 4-1 所示。

表4-1 GATT client 接口说明

接口名称	说明
gattc_register_client	注册 GATT client，同步返回 clientId。

接口名称	说明
gattc_unregister_client	取消注册 GATT client。
gattc_discovery_service	根据服务 UUID 搜索对端 GATT 服务，不传 UUID 时，搜索所有服务。
gattc_discovery_character	根据服务 handle 搜索对端服务特征。
gattc_discovery_descriptor	根据特征 handle 搜索对端服务特征描述符。
gattc_read_req_by_handle	根据 handle 向对端发起读请求。
gattc_read_req_by_uuid	根据 UUID 向对端发起读请求。
gattc_write_req	向对端发起写请求。
gattc_write_cmd	向对端发起写命令，不需要响应。
gattc_exchange_mtu_req	向对端交换 MTU 大小。
gattc_register_callbacks	注册 GATT client 回调函数。

开发流程

GATT client 开发的典型流程：连接对端设备，发现对端设备的服务，读写对端特征值，订阅对端的通知或者指示。

步骤 1 调用 enable_bt_stack，打开蓝牙开关。

步骤 2 调用 gattc_register_callbacks，注册 GATT client 回调。

步骤 3 调用 gattc_register_client，创建一个 client。

步骤 4 调用 gap_ble_connect_remote_device，连接对端设备。

步骤 5 调用 gattc_discovery_service，查找对端支持的服务，支持一次查找所有服务。

步骤 6 调用 gattc_discovery_character，查找指定服务的特征，支持一次查找服务的所有特征。

步骤 7 调用 gattc_discovery_descriptor，查找指定特征的所有描述符。

步骤 8 调用 gattc_read_req_by_handle，根据 handle 读取指定的特征值或描述符值。

步骤 9 调用 gattc_read_req_by_uuid，根据 UUID 读取指定的特征值或描述符值。

- 步骤 10 调用 gattc_write_req, 向对端发起写请求。
- 步骤 11 调用 gattc_write_cmd, 向对端发起写命令, 不需要响应。
- 步骤 12 调用 gattc_exchange_mtu_req, 与对端交换 MTU 大小。

----结束

表4-2 GATT 特征属性定义

序号	定义	实际数值	说明
1	GATT_CHARACTER_PROPERTY_BIT_BROADCAST	0x01	允许广播 characteristic value。
2	GATT_CHARACTER_PROPERTY_BIT_READ	0x02	允许读 characteristic value。
3	GATT_CHARACTER_PROPERTY_BIT_WRITE_NO_RSP	0x04	允许不响应写 characteristic value。
4	GATT_CHARACTER_PROPERTY_BIT_WRITE	0x08	允许写 characteristic value 并且需要给 client 回复。
5	GATT_CHARACTER_PROPERTY_BIT_NOTIFY	0x10	允许把 characteristic value 通知给 client, 并且不需要 client 回复确认。
6	GATT_CHARACTER_PROPERTY_BIT_INDICATE	0x20	允许把 characteristic value 指示给 client, 并且需要 client 回复确认。
7	GATT_CHARACTER_PROPERTY_BIT_SIGNED_WRITE	0x40	允许对 characteristic value 进行签名写入。
8	GATT_CHARACTER_PROPERTY_BIT_EXTENDED_PROPERTY	0x80	在特征扩展特性描述符中定义了附加的特征特性。

表4-3 GATT 操作错误码

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	GATT_STATUS_SUCCESS	0x00	执行成功错误码。
2	GATT_STATUS_INVALID_HANDLE	0x01	无效 handle 错误码。
3	GATT_STATUS_READ_NOT_PERMITTED	0x02	不可读错误码。
4	GATT_STATUS_WRITE_NOT_PERMITTED	0x03	不可写错误码。
5	GATT_STATUS_INVALID_PDU	0x04	无效 PDU 错误码。
6	GATT_STATUS_INSUFFICIENT_AUTHENTICATION	0x05	未认证的读写错误码。
7	GATT_STATUS_REQUEST_NOT_SUPPORTED	0x06	不支持 client 发送的请求。
8	GATT_STATUS_INVALID_OFFSET	0x07	无效偏移的读写错误码。
9	GATT_STATUS_INSUFFICIENT_AUTHORIZATION	0x08	未授权错误码。
10	GATT_STATUS_PREPARE_QUEUE_FULL	0x09	排队的 prepare writes 太多，队列已满错误码。
11	GATT_STATUS_ATTRIBUTE_NOT_FOUND	0x0A	属性不存在错误码。
12	GATT_STATUS_ATTRIBUTE_NOT_LONG	0x0B	该 attribute 不是 long attribute，不能使用 ATT_READ_BLOB_REQ PDU。
13	GATT_STATUS_INSUFFICIENT_ENCRYPTION_KEY_SIZE	0x0C	用于加密此连接的加密密钥大小不足。
14	GATT_STATUS_INVALID_ATTRIBUTE_VALUE_LENGTH	0x0D	无效的属性值长度。
15	GATT_STATUS_UNLIKELY_ERROR	0x0E	不可能的错误码。
16	GATT_STATUS_INSUFFICIENT_ENCRYPTION	0x0F	加密不足错误码。
17	GATT_STATUS_UNSUPPORTED_GROUP_TYPE	0x10	不支持的 grouping attribute 错误码。

序号	定义	实际数值	说明
18	GATT_STATUS_INSUFFICIENT_RESOURCES	0x11	资源不足错误码。
19	GATT_STATUS_DATABASE_OUT_OF_SYNC	0x12	服务端与客户端未同步数据库错误码。
20	GATT_STATUS_VALUE_NOT_ALLOWED	0x13	非法的参数值错误码。

表4-4 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

4.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/ble” 中相关 GATT 头文件实现 GATT 服务发现实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/dis_profile_client.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

- 双模设备可参考以下代码实现：

```
errcode_t ret = enable_bt_stack();  
return ret;
```

- 单模 BLE 设备参考以下代码实现：

```
errcode_t ret = enable_ble();  
if (ret != ERRCODE_BT_SUCCESS) {  
    return BLE_AT_FAIL;  
}  
return BLE_AT_SUCC;
```

步骤 2 实现 GATT 回调函数。

按照 gattc_discovery_service_complete_callback 回调函数为例：

```
void ble_dis_client_discovery_service_cbk(uint8_t client_id, uint16_t conn_id,  
    gattc_discovery_service_result_t *service)  
{  
    /* 发现特征 */  
    gattc_discovery_character_param_t param = {0};  
    param.service_handle = dev->start_hdl;  
    gattc_discovery_character(client_id, conn_id, &param);  
}
```

步骤 3 注册 GATT client 回调函数。

```
gattc_callbacks_t cb = {  
    .discovery_svc_cb = ble_dis_client_discovery_service_cbk,  
    // 其他回调函数按需注册  
};  
errcode_t ret = gattc_register_callbacks(&cb);  
return ret;
```


步骤 4 发现服务。

```
errcode_t ble_dis_client_discovery_service(uint8_t client_id, uint16_t conn_id)
{
    ble_dis_client_remote_dev_t *dev = dis_client_get_dev_by_conn(client_id, conn_id);
    if (dev != NULL) {
        return ERRCODE_BT_FAIL;
    }
    /* 查找服务 */
    bt_uuid_t dis_service_uuid = {0};
    bts_data_to_uuid_len2(BLE_PRO_DIS_SERVICE_UUID16, &dis_service_uuid);
    return gattc_discovery_service(client_id, conn_id, &dis_service_uuid);
}
```

----结束

5 HFP-HF 接口

5.1 概述

5.2 开发流程

5.3 开发实例

5.4 注意事项

5.1 概述

HFP (Hands-Free Profile) 允许耳机、手表、车载免提设备等通过蓝牙连接到手机, 充当手机语音通话的输入输出设备, 同时在不接触手机的情况下完成典型的通话控制操作。

角色划分:

- Hands-Free unit: 免提设备, 如蓝牙耳机、手表、车载免提设备等。
- Audio Gateway: 音频网关, 如手机。

本章节主要是利用 Audio Gateway (音频网关, 如手机) 进行拨打、接听电话等操作的功能对 Hands-Free unit (免提设备) 的相关接口进行介绍。

5.2 开发流程

使用场景

免提设备 (如手表) 与音频网关 (如手机) 相连, 通过远程管理完成典型的通话功能操作, 如接听、拨打、挂断电话和调节音量等。

功能

HFP-HF 功能提供的接口如表 5-1 所示。

表5-1 HFP-HF 功能接口说明

接口名称	说明
hfp_hf_register_callbacks	注册 HFP-HF 回调函数。
hfp_hf_start_dial	拨号发起主叫。
hfp_hf_finish_call	结束通话。
hfp_hf_accept_incoming_call	接听呼入电话。
hfp_hf_reject_incoming_call	拒接呼入电话。
hfp_hf_hold_active_call	挂起/恢复通话。
hfp_hf_send_dtmf_tone	指示 AG 发送 DTMF 码。
hfp_hf_connect	建立与 AG 的服务级别连接。
hfp_hf_disconnect	释放与 AG 的服务级别连接。
hfp_hf_get_device_connect_state	获取设备 Profile 连接状态。
hfp_hf_disconnect_sco	断开与 AG 的 SCO 连接。
hfp_hf_connect_sco	建立与 AG 的 SCO 连接。
hfp_hf_get_sco_connect_state	获取 SCO 连接状态。
hfp_hf_set_volume	HF 端发起音量调节。
hfp_hf_open_voice_recognition	指示 AG 打开语音识别功能。
hfp_hf_close_voice_recognition	指示 AG 关闭语音识别功能。

开发流程

HFP-HF 功能开发的典型流程：

步骤 1 （必要）调用 hfp_hf_register_callbacks，注册 HFP-HF 回调函数。

步骤 2（必要）HF 侧打开蓝牙并开始广播，蓝牙连接到 AG（一般指手机）上。

以下步骤根据需要执行：

步骤 3 调用 hfp_hf_start_dial，可以拨打电话。

步骤 4 调用 hfp_hf_finish_call，可以结束通话。包括结束正在进行和正在拨出的通话，但不包括拒接呼入的电话。

步骤 5 调用 hfp_hf_accept_incoming_call，可以接听呼入的电话。

步骤 6 调用 hfp_hf_reject_incoming_call，可以拒接呼入的电话。

步骤 7 调用 hfp_hf_hold_active_call 可以将 ACTIVE 状态的通话（若存在）转变为 HELD 状态，也可以将 HELD 状态的通话（若存在）转变为 ACTIVE。当同时存在 ACTIVE 和 HELD 的通话时，两路通话的状态会互换。

步骤 8 调用 hfp_hf_send_dtmf_tone 可以在通话中发送按键信息，如在与 10086 等号码通话需要按键时。

步骤 9 调用 hfp_hf_set_volume 可以在通话中调节喇叭音量。

步骤 10 调用 hfp_hf_disconnect_sco 可以断开与 AG 的音频连接。若当前音频在 HF 侧，可以使用该接口转移到 AG 侧。

步骤 11 调用 hfp_hf_connect_sco 可以建立与 AG 的音频连接。若当前音频在 AG 侧，可以使用该接口转移到 HF 侧。

步骤 12 调用 hfp_hf_get_sco_connect_state 可以查询与 AG 的音频连接状态。

步骤 13 调用 hfp_hf_get_device_connect_state 可以查询 Profile 连接状态。

步骤 14 调用 hfp_hf_disconnect 可以释放与 AG 的服务级别连接。

----结束

返回值

返回值请参见对应的接口返回值说明。

hfp_hf_get_device_connect_state 接口返回值如表 5-2 所示。

表5-2 hfp_hf_get_device_connect_state 接口返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

其他函数返回值如表 5-3 所示。

表5-3 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举

序号	定义	实际数值	说明
			最大值。

表5-4 接受一路呼叫时对其他呼叫的操作策略

序号	定义	实际数值	说明
1	HFP_HF_ACCEPT_CALL_ACTION_NONE	0x00	接受呼叫后无操作。
2	HFP_HF_ACCEPT_CALL_ACTION_HOLD	0x01	接受呼叫保持当前通话。
3	HFP_HF_ACCEPT_CALL_ACTION_FINISH	0x02	接受呼叫后结束当前通话。

表5-5 电话呼叫状态类型

序号	定义	实际数值	说明
1	HFP_HF_CALL_STATE_ACTIVE	0	HFP-HF 正在通话。
2	HFP_HF_CALL_STATE_HELD	1	HFP-HF 呼叫保持通话。
3	HFP_HF_CALL_STATE_DIALING	2	HFP-HF 呼叫正在拨号。
4	HFP_HF_CALL_STATE_ALERTING	3	HFP-HF 呼叫正在响铃。
5	HFP_HF_CALL_STATE_INCOMING	4	HFP-HF 呼叫接入。
6	HFP_HF_CALL_STATE_WAITING	5	HFP-HF 呼叫等待。
7	HFP_HF_CALL_STATE_RESPONSE_HELD	6	HFP-HF 呼叫响应保持。

序号	定义	实际数值	说明
8	HFP_HF_CALL_STATE_FINISHED	7	HFP-HF 呼叫结束。

表5-6 HFP 的音量调整类型

序号	定义	实际数值	说明
1	HFP_VOLUME_MIC	0x00	调整 microphone 增益。
2	HFP_VOLUME_SPEAKER	0x01	调整 speaker 增益。

表5-7 codec 编码信息

序号	定义	实际数值	说明
1	HFP_HF_CODEC_ID_CVSD	0x01	HFP-HF 编码格式为 CVSD。
2	HFP_HF_CODEC_ID_MSBC	0x02	HFP-HF 编码格式为 MSBC。

表5-8 hfp_hf_get_sco_connect_state 接口返回值说明

序号	定义	实际数值	说明
1	HFP_SCO_STATE_CONNECTING	0x01	SCO 正在连接。
2	HFP_SCO_STATE_CONNECTED	0x02	SCO 已连接。
3	HFP_SCO_STATE_DISCONNECTING	0x03	SCO 正在断连。
4	HFP_SCO_STATE_DISCONNECTED	0x04	SCO 已断连。

5.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_hfp_hf.h” 实现 HFP-HF 服务设置音量实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_hfp_hf_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();  
return ret;
```

步骤 2 实现 HFP-HF 回调函数。

按照 hfp_hf_volume_changed_callback 回调函数为例：

```
static void sample_handle_hfp_volume_changed(const bd_addr_t *bd_addr, hfp_volume_type_t  
type, uint8_t volume)  
{  
    bts_sample_log_with_func("type: %d, volume: %u\n", type, volume);  
    if (type == HFP_VOLUME_SPEAKER) {  
        sample_hfp_hf_set_spk_volume(volume); // 实现音量下发设置到每天  
    }  
}
```

步骤 3 注册 HFP-HF 回调函数。

```
hfp_hf_callbacks_t cb = {  
    .volume_changed_cb = sample_handle_hfp_volume_changed,  
    // 其他回调函数按需注册  
};  
errcode_t ret = hfp_hf_register_callbacks(&cb);  
return ret;
```

步骤 4 设置音量。

```
uint32_t btsrv_wstp_at_bt_set_spk_vol(const bt_manager_commu_inter_info_t *at_para)  
{  
    int ret = hfp_hf_set_volume(NULL, OHOS_HFP_VOLUME_SPEAKER, at_para->payload[POS_0]);  
    return (uint32_t)ret;  
}
```

----结束

5.4 注意事项

- 调节音量接口 `hfp_hf_set_volume` 只是通知 AG 侧音量发生改变。因此调节音量时还要手动调用媒体接口，否则音量不会真的发生改变。
- 调用 `hfp_hf_connect_sco` 接口将音频切回 HF 时在某些手机可能会失败，需要 AG 侧操作。
- 调用 `hfp_hf_disconnect` 接口断开与 AG 的服务级别连接后，部分 AG 会下发 `detach` 指令，导致蓝牙断开。

6 A2DP-SRC 接口

6.1 概述

6.2 开发流程

6.3 开发实例

6.4 注意事项

6.1 概述

A2DP (Advanced Audio Distribution Profile) 定义了 ACL 信道上进行高质量的音频分发的协议过程。A2DP 角色定义：

- Source：音频源。音频的输入端对音频数据进行编码，通过 ACL 链路发送到 Sink 端。
- Sink：音频接收器。接收到音频数据后，进行解码还原出音频，完成 audio 的数据传输。

该 Profile 主要应用于 BrandyK100 连接蓝牙耳机/音箱播放音乐的场景。在该场景中，BrandyK100 为 Source 角色，蓝牙耳机/音箱端为 Sink 角色。

本章主要介绍 A2DP Source 角色相关接口。

6.2 开发流程

使用场景

蓝牙音频传输场景中的音源端应用开发。

功能

A2DP-SRC 功能提供的接口如表 6-1 所示。

表6-1 A2DP Source 接口说明

接口名称	说明
a2dp_src_get_device_connect_state	获取与指定设备的连接状态。
a2dp_src_get_playing_state	获取与指定设备的播放状态。
a2dp_src_connect	与指定设备端的 A2DP Sink 建立连接。
a2dp_src_disconnect	断开连接。
a2dp_src_start_playing	设置蓝牙音乐的播放为 start 状态。
a2dp_src_suspend_playing	设置蓝牙音乐的播放为 suspend 状态。
a2dp_src_stop_playing	设置蓝牙音乐的播放为 stop 状态。
a2dp_src_get_active_device	获取已建立连接的设备地址。
a2dp_src_register_callbacks	注册 A2DP Source 上层应用回调。
a2dp_src_deregister_callbacks	取消注册 A2DP Source 上层应用回调。

开发流程

音乐播放应用开发的典型流程：

- 步骤 1 调用 a2dp_src_register_callbacks 接口，注册 A2DP Source 上层应用回调函数。
- 步骤 2 调用 gap_br_start_discovery 接口，扫描并与蓝牙音频播放设备（耳机/音箱等）建立 ACL 连接。
- 步骤 3 ACL 连接成功后，初始化音频组件，调用 a2dp_src_connect 连接对端 A2DP Sink 服务。
- 步骤 4 A2DP 连接成功后，通知音频组件蓝牙音频通道已连接（音频组件切换播放通道为蓝牙），请参见《HiDiTingV100 多媒体软件开发指南》的“音频管理-音频流管理”章节内容。

步骤 5 通过音频组件播放音乐（音频组件自动通过蓝牙通道输出音频流），请参见《HiDiTingV100 多媒体软件开发指南》的“音频管理-音乐播放”章节。

----结束

返回值

返回值请参见下表所示的接口返回值说明。

a2dp_src_get_device_connect_state 接口返回值如表 6-2 所示。

表6-2 a2dp_src_get_device_connect_state 接口返回值说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

a2dp_src_get_playing_state 接口返回值如表 6-3 所示。

表6-3 a2dp_src_get_playing_state 接口返回值说明

序号	定义	实际数值	说明
1	A2DP_NOT_PLAYING	0	A2DP 未播放。
2	A2DP_IS_PLAYING	1	A2DP 正在播放。

a2dp_src_get_active_device 接口返回值如表 6-4 所示。

表6-4 a2dp_src_get_active_device 接口返回值说明

序号	定义	实际数值	说明
1	NOT NULL	0	执行成功，返回蓝牙设备的地址。

序号	定义	实际数值	说明
2	NULL	1	执行失败，返回空指针。

表6-5 A2DP 编码格式类型

序号	定义	实际数值	说明
1	A2DP_CODEC_TYPE_SBC	0x00	编码格式类型为 SBC。
2	A2DP_CODEC_TYPE_AAC	0x01	编码格式类型为 AAC。
3	A2DP_CODEC_TYPE_APTX	0x02	编码格式类型为 APTX。
4	A2DP_CODEC_TYPE_APTX_HD	0x03	编码格式类型为 APTX_HD。
5	A2DP_CODEC_TYPE_LDAC	0x04	编码格式类型为 LDAC。
6	A2DP_CODEC_TYPE_LHDC	0x05	编码格式类型为 LHDC。
7	A2DP_CODEC_TYPE_LC3	0x06	编码格式类型为 LC3。
8	A2DP_CODEC_TYPE_L2HC	0x07	编码格式类型为 L2HC。
9	A2DP_CODEC_TYPE_INVALID	0xF4240	无效编码格式类型。

表6-6 A2DP 配置编码格式支持状态

序号	定义	实际数值	说明
1	A2DP_OPTIONAL_CODEC_NOT_SUPP	0x00	不支持设置编码格

序号	定义	实际数值	说明
	ORT		式类型。
2	A2DP_OPTIONAL_CODEC_SUPPORT	0x01	支持设置编码格式类型。
3	A2DP_OPTIONAL_CODEC_SUPPORT_UNKNOWN	0x02	支持用户定义编码格式类型。

表6-7 蓝牙通道参数类型定义

序号	定义	实际数值	说明
1	BT_AUDIO_PARAM_A2DP_CODEC	0x00	A2DP 编码格式。
2	BT_AUDIO_PARAM_AVRCP_VOLUME	0x01	AVRCP 音量。

表6-8 蓝牙音频通道事件类型定义

序号	定义	实际数值	说明
1	BT_AUDIO_A2DP_STREAM_CREATE	0	A2DP 流创建，数据为流 handle，长度为 sizeof(td_pvoid)。
2	BT_AUDIO_A2DP_STREAM_OPENED	1	A2DP 流打开，数据为 bt_audio_a2dp_stream_open_data，长度为 sizeof(td_pvoid)。
3	BT_AUDIO_A2DP_STREAM_STRAT	2	A2DP 流开始，数据为流 handle，长度为 sizeof(td_pvoid)。

序号	定义	实际数值	说明
4	BT_AUDIO_A2DP_STREAM_SUSPENDED	3	A2DP 流暂停，数据为流 handle，长度为 sizeof(td_pvoid)。
5	BT_AUDIO_A2DP_STREAM_CLOSED	4	A2DP 流关闭，数据为流 handle，长度为 sizeof(td_pvoid)。
6	BT_AUDIO_A2DP_STREAM_CONFIG_CHANGE	5	A2DP 流参数变更，数据结构为 bt_audio_a2dp_config_chg_data。
7	BT_AUDIO_SCO_CONN_ESTABLISHED	6	SCO 连接建立，数据为流 handle，长度为 sizeof(td_pvoid)。
8	BT_AUDIO_SCO_CONN_RELEASED	7	SCO 连接释放，数据为流 handle，长度为 sizeof(td_pvoid)。

表6-9 音频端口类型定义

序号	定义	实际数值	说明
1	A2DP	0	A2DP
2	SCO_IN	1	SCO_IN
3	SCO_OUT	2	SCO_OUT
4	BT_AUDIO_PORT_MAX	3	BT_AUDIO_PORT_MAX

其他函数返回值如表 6-10 所示。

表6-10 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

6.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_a2dp_source.h” 实现 A2DP-SRC 服务连接实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_a2dp_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();  
return ret;
```

步骤 2 实现 A2DP-SRC 回调函数。

按照 a2dp_src_connect_state_changed_callback 回调函数为例：

```
static void sample_a2dp_src_connect_state_changed_callback(bd_addr_t *bd_addr, int conn_state)  
{  
    if (conn_state == PROFILE_STATE_CONNECTED) {  
        /* A2DP 连接建立成功后，主动建立AVRCP连接 */  
        int conn_ret = avrcp_ct_connect(bd_addr);  
    }  
}
```

步骤 3 注册 A2DP-SRC 回调函数。

```
a2dp_src_callbacks_t cb = {  
    .connectstate_changed_cb = sample_a2dp_src_connect_state_changed_callback,  
    // 其他回调函数按需注册  
};  
errcode_t ret = a2dp_src_register_callbacks(&cb);  
return ret;
```

步骤 4 连接设备。

```
errcode_t ret = gap_connect_remote_device(bd_addr);  
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

-----结束

6.4 注意事项

- 在本产品中，A2DP-SRC 接口仅用于上层应用控制与蓝牙播放设备的连接和断开，以及音频数据在媒体组件内部通道传输。
- 只支持同时与一个 Sink 设备连接，切换 Sink 设备时，请先断开已连接的 Sink 设备。

7

A2DP-SNK 接口

7.1 概述

7.2 开发流程

7.3 开发实例

7.4 注意事项

7.1 概述

A2DP (Advanced Audio Distribution Profile) 定义了 ACL 信道上进行高质量的音频分发的协议过程。A2DP 角色定义:

- Source: 音频源。音频的输入端对音频数据进行编码, 通过 ACL 链路发送到 Sink 端。
- Sink: 音频接收器。接收到音频数据后, 进行解码还原出音频, 完成 audio 的数据传输。

该 Profile 主要应用于 BrandyK100 连接手机播放手机音乐的场景。在该场景中, 手机为 Source 角色, BrandyK100 为 Sink 角色。

本章主要介绍 A2DP Sink 角色相关接口。

7.2 开发流程

使用场景

蓝牙音频传输场景中的音频接收端应用开发。

功能

A2DP-SNK 功能提供的接口如表 7-1 所示。

表7-1 A2DP Sink 接口说明

接口名称	说明
a2dp_snk_get_device_connect_state	获取与指定设备的连接状态。
a2dp_snk_get_playing_state	获取与指定设备的播放状态。
a2dp_snk_connect	与指定设备端的 A2DP Source 建立连接。
a2dp_snk_disconnect	断开连接。
a2dp_snk_start_playing	设置蓝牙音乐的播放为 start 状态。
a2dp_snk_suspend_playing	设置蓝牙音乐的播放为 suspend 状态。
a2dp_snk_stop_playing	设置蓝牙音乐的播放为 stop 状态。
a2dp_snk_get_active_device	获取已建立连接的设备地址。
a2dp_snk_register_callbacks	注册 A2DP Sink 上层应用回调。
a2dp_snk_deregister_callbacks	取消注册 A2DP Sink 上层应用回调。
a2dp_snk_start_accept	接受启动音频流的请求。
a2dp_snk_service_register	注册 A2DP SNK 服务。
a2dp_snk_service_unregister	去注册 A2DP SNK 服务。

开发流程

音乐播放应用开发的典型流程：

- 步骤 1 调用 a2dp_snk_register_callbacks 接口，注册 A2DP Sink 上层应用回调函数。
- 步骤 2 和手机建立连接。
- 步骤 3 手机端播放音乐。
- 步骤 4 收到手机音乐播放开始请求后，音频组件切换播放通道为蓝牙 Sink，请参见《HiDiTingV100 多媒体软件开发指南》的“音频管理-音频流管理”章节内容。

步骤 5 接受手机音乐播放开始请求，开始播放手机端音频。

----结束

返回值

返回值请参见参见下表所示的接口返回值说明。

a2dp_snk_get_device_connect_state 接口返回值如表 7-2 所示。

表7-2 a2dp_snk_get_device_connect_state 接口返回值说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

a2dp_snk_get_playing_state 接口返回值如表 7-3 所示。

表7-3 a2dp_snk_get_playing_state 接口返回值说明

序号	定义	实际数值	说明
1	A2DP_NOT_PLAYING	0	A2DP 未播放。
2	A2DP_IS_PLAYING	1	A2DP 正在播放。

a2dp_snk_get_active_device 接口返回值如表 7-4 所示。

表7-4 a2dp_snk_get_active_device 接口返回值说明

序号	定义	实际数值	说明
1	NOT NULL	0	执行成功，返回蓝牙设备的地址。
2	NULL	1	执行失败，返回空

序号	定义	实际数值	说明
			指针。

表7-5 A2DP 编码格式类型

序号	定义	实际数值	说明
1	A2DP_SNK_CODEC_TYPE_SBC	0x00	编码格式类型为 SBC。
2	A2DP_SNK_CODEC_TYPE_AAC	0x01	编码格式类型为 AAC。
3	A2DP_SNK_CODEC_TYPE_APTX	0x02	编码格式类型为 APTX。
4	A2DP_SNK_CODEC_TYPE_APTX_HD	0x03	编码格式类型为 APTX_HD。
5	A2DP_SNK_CODEC_TYPE_LDAC	0x04	编码格式类型为 LDAC。
6	A2DP_SNK_CODEC_TYPE_LHDC	0x05	编码格式类型为 LHDC。
7	A2DP_SNK_CODEC_TYPE_LC3	0x06	编码格式类型为 LC3。
8	A2DP_SNK_CODEC_TYPE_L2HC	0x07	编码格式类型为 L2HC。
9	A2DP_SNK_CODEC_TYPE_INVALID	0xF4240	无效编码格式类型。

表7-6 A2DP 配置编码格式支持状态

序号	定义	实际数值	说明
1	A2DP_SNK_OPTIONAL_CODEC_NOT_SUPPORT	0x00	不支持设置编码格式类型。

序号	定义	实际数值	说明
2	A2DP_SNK_OPTIONAL_CODEC_SUPPORT	0x01	支持设置编码格式类型。
3	A2DP_SNK_OPTIONAL_CODEC_SUPPORT_UNKNOWN	0x02	支持用户定义编码格式类型。

表7-7 蓝牙通道参数类型定义

序号	定义	实际数值	说明
1	BT_AUDIO_PARAM_A2DP_CODEC	0x00	A2DP 编码格式。
2	BT_AUDIO_PARAM_AVRCP_VOLUME	0x01	AVRCP 音量。

表7-8 蓝牙音频通道事件类型定义

序号	定义	实际数值	说明
1	BT_AUDIO_A2DP_SNK_STREAM_CREATE	8	A2DP 流创建, 数据为流 handle, 长度为 sizeof(td_pvoid)。
2	BT_AUDIO_A2DP_SNK_STREAM_OPENED	9	A2DP 流打开, 数据为 bt_audio_a2dp_stream_open_data, 长度为 sizeof(td_pvoid)。
3	BT_AUDIO_A2DP_SNK_STREAM_START	10	A2DP 流开始, 数据为流 handle, 长度为 sizeof(td_pvoid)。
4	BT_AUDIO_A2DP_SNK_STREAM_SUSP	11	A2DP 流暂停, 数

序号	定义	实际数值	说明
	ENDED		据为流 handle， 长度为 sizeof(td_pvoid) 。
5	BT_AUDIO_A2DP_SNK_STREAM_CLOSED	12	A2DP 流关闭，数 据为流 handle， 长度为 sizeof(td_pvoid) 。
6	BT_AUDIO_A2DP_SNK_STREAM_CONFIG_CHANGE	13	A2DP 流参数变 更，数据结构为 bt_audio_a2dp_c onfig_chg_data。

表7-9 音频端口类型定义

序号	定义	实际数值	说明
1	A2DP	0	A2DP
2	SCO_IN	1	SCO_IN
3	SCO_OUT	2	SCO_OUT
4	BT_AUDIO_PORT_MAX	3	BT_AUDIO_POR T_MAX

其他函数返回值如表 7-10 所示。

表7-10 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。

序号	定义	实际数值	说明
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

7.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_a2dp_sink.h” 实现 A2DP-SNK 服务连接实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_a2dp_snk_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 实现 A2DP-SNK 回调函数。

按照 a2dp_snk_start_stream_request_callback 回调函数为例：

```
static void sample_a2dp_snk_start_stream_request_callback(bd_addr_t *bd_addr)
{
    AudioManagerA2dpSinkSetParam(g_audio_format_config);
}
```



```
AudioManagerA2dpSinkCreate();  
a2dp_snk_start_accept(&active_bd_addr);  
}
```

步骤 3 注册 A2DP-SNK 回调函数。

```
a2dp_snk_callbacks_t cb = {  
    .start_stream_request_cb = sample_a2dp_src_connect_state_changed_callback,  
    // 其他回调函数按需注册  
};  
errcode_t ret = a2dp_snk_register_callbacks(&cb);  
return ret;
```

步骤 4 与手机建立连接，在手机端播放音乐。

----结束

7.4 注意事项

- 在本产品中，A2DP-SNK 接口仅用于上层应用控制与手机设备的连接和断开，以及音频数据在媒体组件内部通道传输。
- 只支持同时与一个 Source 设备连接，切换 Source 设备时，请先断开已连接的 Source 设备。

8 SPP 接口

- 8.1 概述
- 8.2 开发流程
- 8.3 开发实例
- 8.4 注意事项

8.1 概述

SPP (Serial Port Profile) 定义了设备间基于蓝牙模拟串口通信方式进行数据传输的协议和流程。

本章节介绍 Serial Port Profile 相关接口，主要是 server 端的创建服务器/等待连接/关闭服务器，client 端的连接对端、断开连接，以及双端公用的状态获取和数据读写等接口。

8.2 开发流程

使用场景

设备之间模拟串口通信方式的数据传输。

功能

SPP 功能提供的接口如表 8-1 所示。

表8-1 SPP 接口说明

接口名称	说明
spp_server_create	创建 SPP server。
spp_server_accept	等待远端设备来连接此 server。
spp_server_close	关闭 server（同时释放相关资源）。
spp_connect	连接远端设备。
spp_disconnect	断开连接。
is_spp_connected	获取指定 SPP socket 的连接状态。
spp_get_remote_addr	获取 SPP socket 对应远端设备的地址。
spp_write	向 SPP socket 写入数据。
spp_write_retry_times_config	配置 spp 写失败时重试次数。建议在【enable_bt_stack】之后调用。
spp_register_callbacks	注册 SPP 回调函数

开发流程

SPP server 典型开发流程：

- 步骤 1 调用 spp_server_create 接口，创建一个 SPP server，创建成功后接口返回一个 Server ID。
- 步骤 2 调用 spp_server_accept 接口，等待远端设备来连接步骤 1 中创建的 SPP server，参数传入步骤 1 中返回的 Server ID，连接成功后接口返回一个 Client ID，用于后续状态查询和数据读写。
- 步骤 3 调用 is_spp_connected 接口，传入步骤 2 返回的 Client ID，查询连接状态。
- 步骤 4 调用 spp_get_remote_addr 接口，传入步骤 2 返回的 Client ID，查询对端设备地址。
- 步骤 5 调用 spp_write 接口，传入步骤 2 返回的 Client ID 和待写入数据，向对端设备发送数据。
- 步骤 6 数据通信完成后，若仅希望断开连接，可调用 spp_disconnect 接口，传入步骤 2 返回的 Client ID，仅断开连接而不关闭 server，之后可通过步骤 2 等待连接。

步骤 7 数据通信完成后，若希望断开连接并关闭 server，可调用 spp_server_close 接口，传入步骤 1 返回的 Server ID，断开连接并关闭 server。

----结束

SPP Client 典型开发流程：

- 步骤 1 调用 spp_connect 接口，连接远端设备上的 SPP server，连接成功后接口返回一个 Client ID，用于后续状态查询和数据读写。
- 步骤 2 调用 is_spp_connected 接口，传入步骤 1 返回的 Client ID，查询连接状态。
- 步骤 3 调用 spp_get_remote_addr 接口，传入步骤 1 返回的 Client ID，查询对端设备地址。
- 步骤 4 调用 spp_write 接口，传入步骤 1 返回的 Client ID 和待写入数据，向对端设备发送数据。
- 步骤 5 数据通信完成后，调用 spp_disconnect 接口，传入步骤 1 返回的 Client ID，断开 SPP 连接。

----结束

返回值

返回值请参见对应的接口的返回值说明。

spp_server_create 接口返回值如表 8-2 所示。

表8-2 spp_server_create 接口返回值说明

序号	定义	实际数值	说明
1	SPP_INVALID_ID	-1	执行失败，返回非法值。
2	server_id	0 ~ 15	执行成功，返回服务器 ID。

spp_server_accept 接口返回值如表 8-3 所示。

表8-3 spp_server_accept 接口返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	SPP_INVALID_ID	-1	执行失败，返回非法值。
2	client_id	≥ 0	执行成功，返回等待连接的客户端 ID。

spp_connect 接口返回值如表 8-4 所示。

表8-4 spp_connect 接口返回值说明

序号	定义	实际数值	说明
1	SPP_INVALID_ID	-1	执行失败，返回非法值。
2	client_id	≥ 0	执行成功，返回已连接的客户端 ID。

is_spp_connected 接口返回值如表 8-5 所示。

表8-5 is_spp_connected 接口返回值说明

序号	定义	实际数值	说明
1	true	1	已连接。
2	false	0	未连接。

spp_write 接口返回值如表 8-6 所示。

表8-6 spp_write 接口返回值说明

序号	定义	实际数值	说明
1	SPP_WRITE_FAILED	-1	写数据失败。

序号	定义	实际数值	说明
2	write_len	>0	本次写入数据的长度。

表8-7 SPP socket 类型定义枚举

序号	定义	实际数值	说明
1	SPP_SOCKET_RFCOMM	0x00	RFCOMM。

表8-8 spp 连接状态说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

其他函数返回值如表 8-9 所示。

表8-9 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。

序号	定义	实际数值	说明
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

8.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_spp.h” 实现 SPP 服务写数据实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_spp_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 创建 SPP 服务。

```
spp_create_socket_para_t param = { 0 };
bts_data_to_uuid_len2(0x1101, &param.uuid);
param.socket_type = SPP_SOCKET_RFCOMM;
param.is_encrypt = true;
errcode_t ret = spp_server_create(&param, name, name_len);
if (ret == SPP_INVALID_ID) {
    // 创建Server失败，返回非法ID
}
```

```
// 创建Server成功, 返回Server ID
```

```
return ret;
```

步骤 3 等待远端连接 SPP 服务。

```
int server_id = 0;
```

```
errcode_t ret = spp_server_accept(int server_id);
```

```
if (ret == SPP_INVALID_ID) {
```

```
    // 服务未连接需要继续等待
```

```
}
```

```
// 服务已经连接成功, 返回一个有效的Client ID
```

```
return ret;
```

步骤 4 写 SPP 数据。

```
int client_id = 0; // 使用spp_server_accept返回的Client ID
```

```
char data[10] = { 0 };
```

```
uint32_t data_len = 10;
```

```
errcode_t ret = spp_write(client_id, &data, data_len);
```

```
if (ret == SPP_INVALID_ID) {
```

```
    // 写数据失败, 需要重新写
```

```
}
```

```
// 写数据成功, 返回实际写入长度, 单位字节
```

```
return ret;
```

----结束

8.4 注意事项

- 当前版本 spp_server_create 和 spp_connect 接口中, UUID 字段仅做合法性检查, 暂不会实际生效。若对端设备存在多个 SPP server, 本端调用 spp_connect 将随机连上对端某个 SPP server。后续版本将对此进行优化。
- spp_server_create 和 spp_connect 接口中的 isEncrypt 参数无效, 默认加密。
- 支持通过 spp_server_create 接口创建 16 个 SPP server 以及通过 spp_connect 接口创建 16 个 SPP client, 实际使用过程中, 若每个 server/client 占用两个线程 (读写各一个), 可能会超出系统线程数规格, 需开发过程中酌情处理。

- 接口中的 Server ID 和 Client ID 均是设备本地维护的值，不会在设备间进行交换，因此不同设备间相同的值并无关联意义。

Draft

9

PBAP-PCE 接口

9.1 概述

9.2 开发流程

9.3 开发实例

9.4 注意事项

9.1 概述

PBAP (Phone Book Access Profile) 定义了设备用于检索电话簿对象的协议和程序。其基于客户端-服务器交互模型，其中客户端设备从服务器设备中提取电话簿对象。

角色划分：

- 电话簿服务器设备 (PSE)：包含源电话簿对象的设备，例如：手机。
- 电话簿客户端设备 (PCE)：从服务器设备检索电话簿对象的设备，例如：手表、车载免提设备等。

本章节介绍 PBAP-PCE 相关接口，主要是获取 PSE (如手机) 上存储的联系人、通话记录等功能。

9.2 开发流程

使用场景

PCE 设备 (如手表) 与 PSE (如手机) 相连，PCE 通过调用接口可以获取存储在 PSE 上的联系人、通话记录等信息。

功能

PBAP-PCE 功能提供的接口如表 9-1 所示。

表9-1 PBAP-PCE 功能接口说明

接口名称	说明
pbap_pce_register_callbacks	注册 PBAP-PCE 功能回调函数。
pbap_pce_connect	建立与 PSE 的连接。
pbap_pce_disconnect	断开与 PSE 的连接。
pbap_pce_get_device_connect_state	查询 PCE 连接状态。
pbap_pce_pull_phone_book	下载电话簿对象。
pbap_pce_set_phone_book	设置虚拟文件夹体系结构中的当前文件夹。
pbap_pce_pull_vcard_listing	检索 PSE 的电话簿列表对象。
pbap_pce_pull_vcard_entry	检索特定的 vCard。
pbap_pce_stop_trans	停止传输。

开发流程

PBAP-PCE 功能开发的典型流程：

- 步骤 1（必要）调用 pbap_pce_register_callbacks 接口，注册回调函数。
- 步骤 2（必要）PCE 侧打开蓝牙并开始广播，蓝牙连接到 PSE（一般指手机）上。
- 步骤 3（必要）调用 pbap_pce_connect 接口，可以建立与 PSE 的连接。
- 步骤 4（可选）调用 pbap_pce_pull_phone_book 接口，可以下载电话簿对象。根据需要下载
的类别设置参数，例如：联系人、未接电话、呼入电话、呼出电话、全部通话等。
- 步骤 5（可选）调用 pbap_pce_set_phone_book 接口，可以设置虚拟文件夹体系结构中的当
前文件夹，此步骤是调用 pbap_pce_pullv_card_listing 接口或
pbap_pce_pullv_card_entry 接口之前的必要操作。

- 步骤 6（可选）调用 pbap_pce_pull_vcard_listing 接口，可以按照某种顺序排序并可以根据电话号码或名字检索当前文件夹下 PSE 的电话簿列表对象。
- 步骤 7（可选）调用 pbap_pce_pull_vcard_entry 接口，可以根据步骤 6 返回的 vcf 值，检索当前文件夹下特定的 vCard（单条电话簿详细信息）。
- 步骤 8（可选）调用 pbap_pce_stop_trans 接口，可以停止电话簿数据的传输。
- 步骤 9（可选）调用 pbap_pce_get_device_connect_state 接口，可以查询 PCE 与 PSE 的连接状态。
- 步骤 10（可选）调用 pbap_pce_disconnect 接口，可以断开与 PSE 的连接。

----结束

返回值

返回值请参见下面所示的接口返回值说明。

pbap_pce_get_device_connect_state 接口返回值如表 9-2 所示。

表9-2 pbap_pce_get_device_connect_state 接口返回值说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

表9-3 PBAP vCard 格式取值

序号	定义	实际数值	说明
1	PBAP_FORMAT_VCARD21	0x00	PBAP vCard 格式 v2.1。
2	PBAP_FORMAT_VCARD30	0x01	PBAP vCard 格式 v3.0。

表9-4 PBAP 存储位置

序号	定义	实际数值	说明
1	PBAP_REPOSITORY_LOCAL	0x01	PBAP 电话簿存储位置在手机。
2	PBAP_REPOSITORY_SIM	0x02	PBAP 电话簿存储位置在 SIM 卡。

表9-5 PBAP 排序类型

序号	定义	实际数值	说明
1	PBAP_ORDER_INDEXED	0x00	PBAP 排序顺序为索引序。
2	PBAP_ORDER_ALPHANUMERIC	0x01	PBAP 排序顺序为字典序。
3	PBAP_ORDER_PHONETIC	0x02	PBAP 排序顺序为语音序。

表9-6 PBAP 属性掩码取值

序号	定义	实际数值	说明
1	PBAP_PROPERTY_MASK_VERSION	0x00	PBAP 属性：vCard 版本。
2	PBAP_PROPERTY_MASK_FN	0x01	PBAP 属性：格式化名称。
3	PBAP_PROPERTY_MASK_N	0x02	PBAP 属性：名称的结构化表示。
4	PBAP_PROPERTY_MASK_PHOTO	0x03	PBAP 属性：相关图像或照片。
5	PBAP_PROPERTY_MASK_BDAY	0x04	PBAP 属性：生日。

序号	定义	实际数值	说明
6	PBAP_PROPERTY_MASK_ADR	0x05	PBAP 属性：收货地址。
7	PBAP_PROPERTY_MASK_LABEL	0x06	PBAP 属性：标签。
8	PBAP_PROPERTY_MASK_TEL	0x07	PBAP 属性：电话号码。
9	PBAP_PROPERTY_MASK_EMAIL	0x08	PBAP 属性：电子邮件地址。
10	PBAP_PROPERTY_MASK_MAILER	0x09	PBAP 属性：电子邮件。
11	PBAP_PROPERTY_MASK_RTY_MASK_TZ	0x0A	PBAP 属性：时区。
12	PBAP_PROPERTY_MASK_GEO	0x0B	PBAP 属性：地理位置
13	PBAP_PROPERTY_MASK_TITLE	0x0C	PBAP 属性：工作。
14	PBAP_PROPERTY_MASK_RTY_MASK_ROLE	0x0D	PBAP 属性：组织内的角色。
15	PBAP_PROPERTY_MASK_LOGO	0x0E	PBAP 属性：组织徽标。
16	PBAP_PROPERTY_MASK_RTY_MASK_AGENT	0x0F	PBAP 属性：代表人员的 vCard。
17	PBAP_PROPERTY_MASK_ORG	0x10	PBAP 属性：组织名称。
18	PBAP_PROPERTY_MASK_NOTE	0x11	PBAP 属性：注释。
19	PBAP_PROPERTY_MASK_REV	0x12	PBAP 属性：修订版本。

序号	定义	实际数值	说明
20	PBAP_PROPERTY_MASK_SOUND	0x13	PBAP 属性：姓名读音。
21	PBAP_PROPERTY_MASK_URL	0x14	PBAP 属性：统一资源定位器。
22	PBAP_PROPERTY_MASK_UID	0x15	PBAP 属性：唯一标识。
23	PBAP_PROPERTY_MASK_KEY	0x16	PBAP 属性：公共加密密钥。
24	PBAP_PROPERTY_MASK_NICKNAME	0x17	PBAP 属性：昵称。
25	PBAP_PROPERTY_MASK_RTY_MASK_RTY_MASK_CATEGORIES	0x18	PBAP 属性：类别。
26	PBAP_PROPERTY_MASK_RTY_MASK_PROID	0x19	PBAP 属性：产品 ID。
27	PBAP_PROPERTY_MASK_CLASS	0x1A	PBAP 属性：CLASS 信息。
28	PBAP_PROPERTY_MASK_RTY_MASK_RTY_MASK_SORT_STRING	0x1B	PBAP 属性：用于排序操作的字符串。
29	PBAP_PROPERTY_MASK_X_IRMC_CALL_DATETIME	0x1C	PBAP 属性：时间戳。
30	PBAP_PROPERTY_MASK_RTY_MASK_X_BT_SPEEDDIALKEY	0x1D	PBAP 属性：快速拨号快捷方式。
31	PBAP_PROPERTY_MASK_RTY_MASK_X_BT_UCI	0x1E	PBAP 属性：统一主叫方标识符。
32	PBAP_PROPERTY_MASK_X_BT_UID	0x1F	PBAP 属性：蓝牙联系人唯一标识符。
33	PBAP_PROPERTY_MASK_FILTER	0x27	PBAP 属性：私有

序号	定义	实际数值	说明
			过滤器。
34	PBAP_PROPERTY_MASK_INVALID	0xFF	-

表9-7 PBAP 错误码

序号	定义	实际数值	说明
1	PBAP_ERROR_OK	0x100	PBAP 错误码：正常。
2	PBAP_ERROR_BAD_REQUEST	0x101	PBAP 错误码：错误请求。
3	PBAP_ERROR_NOT_IMPLEMENTED	0x102	PBAP 错误码：未执行。
4	PBAP_ERROR_UNAUTHORIZED	0x103	PBAP 错误码：未授权。
5	PBAP_ERROR_PRECONDITION_FAILED	0x104	PBAP 错误码：预置条件失败。
6	PBAP_ERROR_NOT_FOUND	0x105	PBAP 错误码：未找到。
7	PBAP_ERROR_NOT_ACCEPTABLE	0x106	PBAP 错误码：不可接受。
8	PBAP_ERROR_SERVICE_UNAVAILABLE	0x107	PBAP 错误码：服务不可用。
9	PBAP_ERROR_FORBIDDEN	0x108	PBAP 错误码：禁止。

其他函数返回值如表 9-8 所示。

表9-8 其他函数返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

9.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_pbap.h” 实现 PBAB-PCE 服务检索 PSE 的电话簿列表对象实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_pbap_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 实现 PBAP-PCE 回调函数。

按照 pbap_pce_receive_data_callback 回调函数为例：

```
static void sample_pbap_receive_data(const bd_addr_t *bd_addr, const uint8_t* data, int32_t len)
{
    // 对接收到的数据进行处理
    return;
}
```

步骤 3 注册 PBAP-PCE 回调函数。

```
pbap_pce_callbacks_t cb = {
    .receive_data_cb = sample_pbap_receive_data,
    // 其他回调函数按需注册
};
errcode_t ret = pbap_pce_register_callbacks(&cb);
return ret;
```

步骤 4 蓝牙连接到 PSE（一般指手机）上，PSE 上设置共享联系人。**步骤 5 建立与 PSE 的连接。**

```
errcode_t ret = pbap_pce_connect(bd_addr);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

步骤 6 设置虚拟文件夹体系结构中的当前文件夹。

```
errcode_t ret pbap_pce_set_phone_book(NULL, "telecom");
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

步骤 7 检索 PSE 的电话簿列表对象。

```
unsigned char *folder = "pb";
pbap_pce_pull_vcard_listing_param_t thr_par = { 0 };
thr_par.name = folder;
thr_par.max_list_count = 65535; /* 最大列表计数，最大65535根据业务设置 */
thr_par.list_offset = 0; /* 列表起始偏移量 */
thr_par.order = order; /* 0:索引序 1:字典序 */
errcode_t ret = pbap_pce_pull_vcard_listing(NULL, &thr_par);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

步骤 8 根据电话号码或名字检索电话簿列表对象。

```
unsigned char *folder = "pb";
```

```
pbap_pce_pull_vcard_listing_param_t thr_par = { 0 };
thr_par.name = folder;
thr_par.max_list_count = 1; /* 最大列表计数 */
thr_par.list_offset = 0; /* 列表起始偏移量 */
thr_par.order = order; /* 0:索引序 1:字典序 */
thr_par.search_value = (unsigned char*)number; /* 电话号码或名字, 查询条件要有字符串结束符 */
thr_par.search_property = PBAP_SEARCH_PROPERTY_NUMBER; /*
PBAP_SEARCH_PROPERTY_NAME:根据名字查询, PBAP_SEARCH_PROPERTY_NUMBER:根据
电话号码查询 */
errcode_t ret = pbap_pce_pull_vcard_listing(NULL, &thr_par);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

-----结束

9.4 注意事项

- 由于不同手机对 PSE 的实现不尽相同, 因此获取某些类别的电话簿可能没有数据, 例如: 速拨联系人、收藏联系人、SIM 卡通话记录等。
- 一条电话簿数据可能包含多个属性, 获取时选择需要的属性即可, 若不选则默认返回所有属性 (包括联系人照片等数据量大但可能无用的属性)。
- 与手机进行 PBAP 连接时, 需要手机端点击确认, 如果用户未点击会阻塞其他蓝牙业务的进行。
- 建议上层应用适配时, 提前定时同步电话簿, 不要在通话来电时实时拉取。

10 AVRCP-CT 接口

10.1 概述

10.2 开发流程

10.3 开发实例

10.1 概述

AVRCP (Audio/Video Remote Control Profile) 描述了在蓝牙设备间实现音频和视频的控制功能。

角色划分：

- controller (CT)：发送命令控制 TG 的设备，例如：耳机。
- target (TG)：接收 CT 发送过来的指令，同时也能通知 CT 一些状态的改变，例如：音箱。

本章节介绍 AVRCP-CT 相关接口，主要是创建连接、断开连接、状态获取、事件查询、事件注册和各种按键功能实现的接口。

10.2 开发流程

使用场景

蓝牙音频传输场景中的 AVRCP-CT 端应用开发，例如：控制 TG 端音乐播放暂停、音量调整和绝对音量。

功能

AVRCP-CT 功能提供的接口如表 10-1 所示。

表10-1 AVRCP-CT 接口说明

接口名称	说明
avrcp_ct_get_device_connect_state	获取与指定设备的 AVRCP-CT 连接状态。
avrcp_ct_get_play_status	获取 TG 设备的播放状态。
avrcp_ct_connect	CT 端发起建立 AVRCP 连接。
avrcp_ct_disconnect	CT 发起断开 AVRCP 连接。
avrcp_ct_press_button	通知 TG 端 CT 已经按下 button。
avrcp_ct_release_button	通知 TG 端 CT 已经释放 button。
avrcp_ct_get_supported_companies	获取 TG 端支持的公司列表。
avrcp_ct_get_supported_events	获取 TG 端支持的事件列表。
avrcp_ct_get_element_attributes	获取 TG 端支持的属性列表，例如：歌曲标题、专辑、艺术风格、年份和作曲家等。
avrcp_ct_register_notification	CT 端向 TG 端注册事件。
avrcp_ct_set_absolute_volume	设置 TG 端的绝对音量。
avrcp_ct_register_callbacks	注册 AVRCP-CT 上层应用回调函数。

开发流程

音乐播放应用开发的典型流程：

- 步骤 1 调用 avrcp_ct_register_callbacks，注册 AVRCP-CT 上层应用回调函数。
- 步骤 2 通过 avrcp_ct_connect 建立 AVRCP 连接。
- 步骤 3 AVRCP 连接成功后，通过回调上报上层应用 AVRCP 已连接。

- 步骤 4 调用 `avrcp_ct_get_supported_events`，获取 TG 端支持的事件列表，事件类型请参见表 10-4。
- 步骤 5 调用 `avrcp_ct_register_notification`，注册 CT 端和 TG 端均支持的事件。
- 步骤 6 注册 `AVRCP_NOTIFY_EVENT_PLAYBACK_STATUS_CHANGED` 事件，控制 TG 端的播放状态。
- 步骤 7 调用 `avrcp_ct_press_button`，通知 TG 端已按下播放按键，TG 端响应 CT 端。
- 步骤 8 调用 `avrcp_ct_release_button`，通知 TG 端已释放播放按键，TG 端响应播放，并向 CT 端返回当前播放状态。
- 步骤 9 CT 端收到 TG 端返回的播放状态后，再次调用 `avrcp_ct_get_supported_events`，注册播放变更事件以支持下一次控制。
- 步骤 10 调用 `avrcp_ct_disconnect`，断开 AVRCP 连接。

----结束

返回值

返回值请参见对应接口的返回值说明。

`avrcp_ct_get_device_connect_state` 接口返回值如表 10-2 所示。

表10-2 `avrcp_ct_get_device_connect_state` 接口返回值说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

其他函数返回值如表 10-3 所示。

表10-3 其他函数返回值说明

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

AVRCP 通知事件类型如表 10-4 所示。

表10-4 AVRCP 通知事件类型值说明

序号	定义	实际数值	说明
1	AVRCP_NOTIFY_EVENT_PLAYBACK_STATUS_CHANGED	1	播放状态变更事件。
2	AVRCP_NOTIFY_EVENT_TRACK_CHANGED	2	track 状态变更事件。

序号	定义	实际数值	说明
3	AVRCP_NOTIFY_EVENT_TRACK_REACHED_END	3	track 结束事件。
4	AVRCP_NOTIFY_EVENT_TRACK_REACHED_START	4	track 开始事件。
5	AVRCP_NOTIFY_EVENT_PLAYBACK_POSITION_CHANGED	5	播放位置改变事件。
6	AVRCP_NOTIFY_EVENT_BATTERY_STATUS_CHANGED	6	电池状态变更事件。
7	AVRCP_NOTIFY_EVENT_SYSTEM_STATUS_CHANGED	7	系统状态变更事件。
8	AVRCP_NOTIFY_EVENT_PLAYER_APPLICATION_SETTING_CHANGED	8	播放器配置变更事件。
9	AVRCP_NOTIFY_EVENT_NOW_PLAYING_CONTENT_CHANGED	9	播放内容变更事件。
10	AVRCP_NOTIFY_EVENT_AVAILABLE_PLAYERS_CHANGED	10	可使用的播放器变更事件。
11	AVRCP_NOTIFY_EVENT_ADDRESSED_PLAYER_CHANGED	11	播放地址变更事件。
12	AVRCP_NOTIFY_EVENT_UIDS_CHANGED	12	UID 变更事件。
13	AVRCP_NOTIFY_EVENT_VOLUME_CHANGED	13	TG 端音量变更事件。
14	AVRCP_NOTIFY_EVENT_RESERVED	14	保留。

表10-5 AVRCP 的按键操作

序号	定义	实际数值	说明
1	AVRCP_KEY_VOLUME_UP	0x41	音量调大操作。
2	AVRCP_KEY_VOLUME_DOWN	0x42	音量调小操作。

序号	定义	实际数值	说明
3	AVRCP_KEY_MUTE	0x43	静音操作。
4	AVRCP_KEY_PLAY	0x44	play 操作。
5	AVRCP_KEY_STOP	0x45	stop 操作。
6	AVRCP_KEY_PAUSE	0x46	pause 操作。
7	AVRCP_KEY_RECORD	0x47	record 操作。
8	AVRCP_KEY_REWIND	0x48	媒体设置为从起始位置开始。
9	AVRCP_KEY_FAST_FORWARD	0x49	设置媒体快进。
10	AVRCP_KEY_FORWARD	0x4B	下一首。
11	AVRCP_KEY_BACKWARD	0x4C	上一首。
12	AVRCP_KEY_INVALID	0x7F	未知操作。

说明

取值的定义参考规范：1394 Trade Association。

表10-6 AVRCP 元素属性 ID

序号	定义	实际数值	说明
1	AVRCP_CT_ELEMENT_ATTR_TITLE	0x01	标题，歌曲名或者内容描述。
2	AVRCP_CT_ELEMENT_ATTR_ARTIST_NAME	0x02	艺人、表演者或团体。
3	AVRCP_CT_ELEMENT_ATTR_ALBUM_NAME	0x03	唱片，专辑。
4	AVRCP_CT_ELEMENT_ATTR_TRACK_NUMBER	0x04	音频文件的原始编号。
5	AVRCP_CT_ELEMENT_ATTR_TOTAL_NUMBER_OF_TRACKS	0x05	tracks/elements 的总个数。

序号	定义	实际数值	说明
6	AVRCP_CT_ELEMENT_ATTR_TRACK_GENRE	0x06	风格, 流派。
7	AVRCP_CT_ELEMENT_ATTR_TRACK_PLAYING_TIME	0x07	音频文件的播放时间。

表10-7 绝对音量范围 (有效的取值范围[0x00, 0x7F], 表示 0% ~ 100%)

序号	定义	实际数值	说明
1	AVRCP_ABSOLUTE_VOLUME_PERCENTAGE_0	0x00	绝对音量 0%。
2	AVRCP_ABSOLUTE_VOLUME_PERCENTAGE_100	0x7F	绝对音量 100%。
3	AVRCP_ABSOLUTE_VOLUME_INVALID	0x80	无效值。

表10-8 AVRCP 媒体播放状态

序号	定义	实际数值	说明
1	AVRCP_PLAY_STATUS_STOPPED	0x00	AVRCP 媒体 stopped 状态。
2	AVRCP_PLAY_STATUS_PLAYING	0x01	AVRCP 媒体 playing 状态。
3	AVRCP_PLAY_STATUS_PAUSED	0x02	AVRCP 媒体 Paused 状态。
4	AVRCP_PLAY_STATUS_FWD_SEEK	0x03	AVRCP 媒体切换下一首。
5	AVRCP_PLAY_STATUS_REV_SEEK	0x04	AVRCP 媒体切换上一首。
6	AVRCP_PLAY_STATUS_ERROR	0xFF	AVRCP 媒体未知状态。

表10-9 AVRCP 电量状态

序号	定义	实际数值	说明
1	AVRCP_BATTERY_STATUS_NORMAL	0x00	电池的电量处于正常状态。
2	AVRCP_BATTERY_STATUS_WARNING	0x01	电池的电量处于告警状态。
3	AVRCP_BATTERY_STATUS_CRITICAL	0x02	电池的电量处于红色告警状态，不能再使用。
4	AVRCP_BATTERY_STATUS_EXTERNAL	0x03	插接外部电源。
5	AVRCP_BATTERY_STATUS_FULL_CHARGE	0x04	电池已充满电。

表10-10 AVRCP 系统状态

序号	定义	实际数值	说明
1	AVRCP_CT_SYSTEM_STATUS_POWER_ON	0x00	AVRCP 开机事件。
2	AVRCP_CT_SYSTEM_STATUS_POWER_OFF	0x01	AVRCP 关机事件。
3	AVRCP_CT_SYSTEM_STATUS_UNPLUGGED	0x02	AVRCP 未充电状态。

10.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_avrcp_controller.h” 实现 AVRCP-CT 服务实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_avrcp_service.c” 代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
```

```
return ret;
```

步骤 2 实现 AVRCP-CT 回调函数。

按照 `avrcp_ct_conn_state_changed_callback` 回调函数为例：

```
static void sample_avrcp_ct_connect_state_changed_callback(const bd_addr_t *bd_addr,
profile_connect_state_t state)
{
    if (bd_addr == NULL) {
        // 空指针处理

        return;
    }

    // 上层应用处理连接状态变化
}
```

步骤 3 注册 AVRCP-CT 回调函数。

```
avrcp_ct_callbacks_t cb = {
    .conn_state_changed_cb = avrcp_ct_conn_state_changed_callback,
    // 其他回调函数按需注册
};
errcode_t ret = avrcp_ct_register_callbacks(&cb);
return ret;
```

步骤 4 连接 AVRCP-CT 服务。

```
errcode_t ret = avrcp_ct_connect(bd_addr);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

----结束

11

AVRCP-TG 接口

11.1 概述

11.2 开发流程

11.3 开发实例

11.4 注意事项

11.1 概述

AVRCP (Audio/Video Remote Control Profile) 描述了在蓝牙设备间实现音频和视频的控制功能。

角色划分：

- controller (CT)：发送命令控制 TG 的设备，例如：耳机。
- target (TG)：接收 CT 发送过来的指令，同时也能通知 CT 一些状态的变化，例如：音箱。

本章节介绍 AVRCP-TG 相关接口，主要是创建连接、断开连接、状态获取、能力集注册，通知 CT 端 TG 端的状态等的接口。

11.2 开发流程

使用场景

蓝牙音频传输场景中的 AVRCP-TG 端应用开发，例如：CT 控制 TG 端音乐播放暂停、音量调整和绝对音量。

功能

AVRCP-TG 功能提供的接口如表 11-1 所示。

表11-1 AVRCP-TG 接口说明

接口名称	说明
avrcp_tg_get_device_connect_state	获取与指定设备的 AVRCP-TG 连接状态。
avrcp_tg_connect	TG 端发起建立 AVRCP 连接。
avrcp_tg_disconnect	TG 端发起断开 AVRCP 连接。
avrcp_tg_notify_playback_status_changed	通知 CT 端播放状态发生改变。
avrcp_tg_notify_track_changed	通知 CT 端 track 发生改变。
avrcp_tg_notify_track_reached_end	通知 CT 端 track 已播放结束。
avrcp_tg_notify_track_reached_start	通知 CT 端 track 已播放开始。
avrcp_tg_notify_playback_position_changed	通知 CT 端播放位置发生改变。
avrcp_tg_notify_now_playing_content_changed	通知 CT 端 NowPlaying folder 发生改变。
avrcp_tg_notify_available_players_changed	通知 CT 端有新的播放器可用。
avrcp_tg_notify_addressed_players_changed	通知 CT 端 Addressed player 发生改变。
avrcp_tg_notify_uid_changed	通知 CT 端 UID 发生改变。
avrcp_tg_notify_volume_changed	TG 端通知 CT 端音量发生改变。
avrcp_tg_register_callbacks	注册 AVRCP-TG 上层应用回调函数。

开发流程

音乐播放应用开发的典型流程：

- 步骤 1 调用 avrcp_tg_register_callbacks，注册 AVRCP-TG 上层应用回调函数。
- 步骤 2 通过 avrcp_tg_connect 建立 AVRCP 连接。

- 步骤 3 AVRCP 连接成功后，通过回调上报上层应用 AVRCP 已连接。
- 步骤 4 当 TG 端音量改变时，调用 `avrcp_tg_notify_volume_changed`，通知 CT 端音量发生改变。
- 步骤 5 当 TG 端播放状态改变时，调用 `avrcp_tg_notify_playback_status_changed`，通知 CT 端播放状态发生改变。
- 步骤 6 调用 `avrcp_tg_disconnect`，断开 AVRCP 连接。

----结束

返回值

返回值请参见对应接口的返回值说明。

`avrcp_tg_get_device_connect_state` 接口返回值如表 11-2 所示。

表11-2 `avrcp_tg_get_device_connect_state` 接口返回值说明

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

其他函数返回值如表 11-3 所示。

表11-3 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。

序号	定义	实际数值	说明
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

11.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_avrcp_target.h” 实现 AVRCP-TG 服务连接实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_avrcp_service.c” 代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 实现 AVRCP-TG 回调函数。

按照 avrcp_tg_conn_state_changed_callback 回调函数为例：

```
static void sample_avrcp_tg_connect_state_changed_callback(const bd_addr_t *bd_addr,
profile_connect_state_t state)
{
    if (bd_addr == NULL) {
```



```
// 空指针处理
return;
}
// 上层应用处理连接状态变化
}
```

步骤 3 注册 AVRCP-TG 回调函数。

```
avrcp_tg_callbacks_t cb = {
    .conn_state_changed_cb = avrcp_ct_conn_state_changed_callback,
    // 其他回调函数按需注册
};
errcode_t ret = avrcp_ct_register_callbacks(&cb);
return ret;
```

步骤 4 连接 AVRCP-TG 服务。

```
errcode_t ret = avrcp_tg_connect(bd_addr);
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

----结束

11.4 注意事项

TG 可能会收到 CT 发送的大量音量设置消息，TG 处理不过来时会导致设备异常，所以对 TG 收到的音量设置消息进行了合并处理。当前策略是每 200ms 内的消息合并成一条处理，每次只处理 200ms 内的最后一条音量设置消息，因此音量设置存在 200ms 延时。

12 PAN-PANU 接口

12.1 概述

12.2 开发流程

12.3 开发实例

12.1 概述

PAN (Personal Area Networking) 描述了在蓝牙设备间通过蓝牙网络共享实现上网功能。本服务仅作为数据通道使用，不提供 TCP/IP 协议功能。

角色划分：

- PANU (PAN User)：发起 pan 服务连接的设备，作为客户端，本身不具备上网功能，只具有直接通信功能，不能转发网络数据包，例如：手表。
- NAP (Network Access Points)：作为服务端，具有上网功能，通过蓝牙网络共享，可以被 PANU 连接，例如：手机。

本章节介绍 PAN-PANU 相关接口，主要是回调注册、取消回调注册、创建连接、断开连接、获取连接状态、发送网络数据包的接口。

12.2 开发流程

使用场景

通过蓝牙共享网络的场景使用，手表作为 PANU 角色，可以上网的手机作为 NAP 角色，手机开启蓝牙网络共享，手表连接手机的 pan 服务，通过网络共享，实现上网功能。

功能

本服务仅作为数据通道使用，不提供 TCP/IP 协议功能。

PAN-PANU 功能提供的接口如表 12-1 所示。

表12-1 PAN-PANU 接口说明

接口名称	说明
pan_register_callbacks	注册回调。
pan_deregister_callbacks	取消注册回调。
pan_service_open	发起 pan 服务连接。
pan_service_close	断开 pan 服务连接。
pan_service_write_data	发送网络数据包。
pan_service_net_state_get	获取连接状态。

开发流程

通过蓝牙实现网络共享开发的典型流程：

- 步骤 1 调用 pan_register_callbacks，注册上层应用回调函数。例如接收网络数据包，连接状态变更通知。
- 步骤 2 调用 pan_service_open 手表主动建立 pan 服务连接。
- 步骤 3 pan 服务连接成功后，通过回调上报上层应用 pan 已连接。
- 步骤 4 pan 服务连接成功后，当对端发过来网络数据包时，通过注册的回调函数通知上层应用。
- 步骤 5 pan 服务连接成功后，上层应用调用 pan_service_write_data，可以向对端发送网络数据包。
- 步骤 6 调用 pan_service_net_state_get，可以检测 pan 服务连接状态。
- 步骤 7 调用 pan_deregister_callbacks，可以取消回调注册。
- 步骤 8 调用 pan_service_close，可以断开 pan 服务连接。

----结束

返回值

返回值请参见对应接口的返回值说明。

pan_service_net_state_get 接口返回值如表 12-2 所示。

表12-2 pan_service_net_state_get 接口返回值说明

序号	定义	实际数值	说明
1	PAN_NET_STATE_OFF	0	离线
2	PAN_NET_STATE_ON	1	在线

其他函数返回值如表 12-3 所示。

表12-3 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。

序号	定义	实际数值	说明
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

表12-4 发送网络数据包协议类型

序号	定义	实际数值	说明
1	PAN_NETWORK_PROTOCOL_IPV4	0x0800	发送 IPv4 数据包。
2	PAN_NETWORK_PROTOCOL_ARP	0x0806	发送 ARP 数据包。

表12-5 发送数据包报文类型

序号	定义	实际数值	说明
1	PAN_PACKET_TYPE_DEST_ONLY_UNICAST	0	报文类型是保留目的地址的单播。
2	PAN_PACKET_TYPE_DEST_ONLY_BROADCAST	1	报文类型是保留目的地址的广播。
3	PAN_PACKET_TYPE_GENERAL_UNICAST	2	报文类型是通用类型的单播。
4	PAN_PACKET_TYPE_GENERAL_BROADCAST	3	报文类型是通用类型的广播。
5	PAN_PACKET_TYPE_COMPRESSED	4	报文类型是压缩头报文。

12.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_pan.h” 实现 PAN-PANU 服务向对端发送网络数据包实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_pan_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();  
return ret;
```

步骤 2 实现 PAN-PANU 回调函数。

按照 pan_net_state_callback 回调函数为例：

```
static void sample_pan_net_state_callback(uint8_t net_state)  
{  
    // 上层业务处理收到的HF端接听请求  
}
```

步骤 3 注册 PAN-PANU 回调函数。

```
hfp_ag_callbacks_t cb = {  
    .answer_call_req_cb = sample_pan_net_state_callback,  
    // 其他回调函数按需注册  
};  
errcode_t ret = pan_register_callbacks(&cb);  
return ret;
```

步骤 4 建立 pan 服务连接。

```
errcode_t ret = pan_service_open(bd_addr);  
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

步骤 5 向对端发送网络数据包。

```
int ip_packet_len = 60;  
uint8_t ip_packet[60] = {  
    0x45, 0x00, 0x00, 0x3c, 0x4c, 0x8d, 0x00, 0x00, 0x80, 0x01, 0x00, 0x00,  
    0xc0, 0xa8, 0x2c, 0x01,  
    0x0e, 0x77, 0x68, 0xfe,  
    0x08, 0x00, 0x4c, 0x26, 0x00, 0x01, 0x01, 0x35, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66,  
    0x67, 0x68, 0x69, 0x6a, 0x6b, 0x6c, 0x6d, 0x6e, 0x6f, 0x70, 0x71, 0x72, 0x73, 0x74, 0x75,  
    0x76,  
    0x77, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67, 0x68, 0x69
```

```
};  
errcode_t ret = pan_service_write_data(ip_packet, ip_packet_len,  
PAN_NETWORK_PROTOCOL_IPV4, PAN_PACKET_TYPE_DEST_ONLY_UNICAST);  
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

----结束

Draft

13 HFP-AG 接口

13.1 概述

13.2 开发流程

13.3 开发实例

13.1 概述

HFP (Hands-Free Profile) 允许耳机、手表、车载免提设备等通过蓝牙连接到手机，充当手机语音通话的输入输出设备，同时在不接触手机的情况下完成典型的通话控制操作。

角色划分：

- Hands-Free unit (HF)：免提设备，如蓝牙耳机、手表、车载免提设备等。
- Audio Gateway (AG)：音频网关，如手机，支持 SIM 卡通话的手表等。

本章节主要是介绍 Audio Gateway (AG) 的功能接口。

13.2 开发流程

📖 说明

HFP-AG 主动连接 HF 时，例如蓝牙耳机，连接成功后，耳机会发送“AT+CIND?”请求，AG 无响应的话，耳机会断开连接，所以 AG 侧收到 HF 侧当前指示器信息请求“AT+CIND?”回调时，要及时调用 `hfp_ag_send_indicator_val` 接口响应。

使用场景

免提设备（如耳机）与音频网关（如支持 SIM 卡的手表）相连，通过远程管理完成典型的通话功能操作，如接听、挂断电话，调节音量，语音通话等。

以下功能暂不支持：

- 三方通话。
- 语音识别。
- 设备同时充当 HFP-HF 和 HFP-AG 角色进行蓝牙通话。

功能

HFP-AG 功能提供的接口如表 13-1 所示。

表13-1 HFP-AG 功能接口说明

接口名称	说明
hfp_ag_connect	建立 HFP profile 连接。
hfp_ag_disconnect	断开 HFP profile 连接。
hfp_ag_audio_connect	建立 HFP 音频连接。
hfp_ag_audio_disconnect	断开 HFP 音频连接。
hfp_ag_get_device_connect_state	获取 HFP profile 连接状态。
hfp_ag_get_sco_connect_state	获取 HFP 音频连接状态。
hfp_ag_send_answer_call_event	AG 接听电话通知。
hfp_ag_send_finish_call_event	挂断或拒接通知。
hfp_ag_send_originate_call_event	AG 呼出电话通知。
hfp_ag_send_network_incoming_event	发送来电事件。
hfp_ag_send_network_event	发送网络事件。
hfp_ag_send_current_calls	发送通话信息。
hfp_ag_send_subscriber_number	发送订阅号码信息。
hfp_ag_send_battery_charge	发送电池电量变化值。

接口名称	说明
hfp_ag_send_cmee_error	发送错误码。
hfp_ag_send_network_operator	发送运营商信息。
hfp_ag_set_volume	调节对端扬声器或麦克风音量大小。
hfp_ag_send_indicator_val	发送指示器信息值。
hfp_ag_register_callbacks	注册 HFP-AG 回调。

开发流程

HFP-AG 功能开发的典型流程：

前提条件：

步骤 1（必要）AG 侧调用 enable_bt_stack 接口，BT 使能。

步骤 2（必要）AG 侧调用 hfp_ag_register_callbacks，注册 HFP-AG 回调函数。

步骤 3（必要）HF 侧打开蓝牙并开始广播，AG 侧设备调用 gap_connect_remote_device 接口，进行蓝牙连接。

步骤 4（必要）AG 侧调用 hfp_ag_connect 接口，与 HF 设备进行 HFP profile 连接。

----结束

以下步骤根据不同的场景执行。

场景一：AG 拨出电话场景

步骤 1 调用 hfp_ag_send_originate_call_event，AG 呼出电话通知耳机。

步骤 2 对端接听后，调用 hfp_ag_send_network_event 接口，接听事件通知 HF 侧。

步骤 3 对端拒接后，调用 hfp_ag_send_finish_call_event，拒接事件通知 HF 侧。

步骤 4 对端挂断后，调用 hfp_ag_send_finish_call_event，挂断事件通知 HF 侧。

步骤 5 AG 侧挂断后，调用 hfp_ag_send_finish_call_event，挂断事件通知 HF 侧。

----结束

场景二：AG 来电场景

- 步骤 1 调用 hfp_ag_send_network_incoming_event，来电事件通知 HF。
- 步骤 2 AG 接听后，调用 hfp_ag_send_answer_call_event，接听事件通知 HF。
- 步骤 3 AG 拒绝后，调用 hfp_ag_send_finish_call_event，拒接事件通知 HF 侧。
- 步骤 4 AG 挂断后，调用 hfp_ag_send_finish_call_event，挂断事件通知 HF 侧。

----结束**场景三：AG 侧请求调整 HF 侧麦克风或扬声器音量大小**

- 步骤 1 调用 hfp_ag_set_volume 接口，调节 HF 侧麦克风或扬声器大小。

----结束**场景四：AG 侧收到 HF 侧的请求**

- 步骤 1 AG 侧收到 HF 侧订阅号码信息请求 (AT+CNUM)，调用 hfp_ag_send_subscriber_number 接口或 hfp_ag_send_cmee_error 接口响应。
- 步骤 2 AG 侧收到 HF 侧当前通话列表请求 (AT+CLCC)，调用 hfp_ag_send_current_calls 接口或 hfp_ag_send_cmee_error 接口响应。
- 步骤 3 AG 侧收到 HF 侧运营商信息请求 (AT+COPS=) (AT+COPS?)，调用 hfp_ag_send_network_operator 接口或 hfp_ag_send_cmee_error 接口响应。
- 步骤 4 AG 侧收到 HF 侧挂断请求 (AT+CHUP)，调用 hfp_ag_audio_disconnect 接口响应。
- 步骤 5 AG 侧收到 HF 侧当前指示器信息请求 (AT+CIND?)，调用 hfp_ag_send_indicator_val 接口响应。

----结束**场景五：AG 侧其他行为**

- 步骤 1 AG 侧主动发送电量变化，调用 hfp_ag_send_battery_charge 接口，通知 HF 侧。
- 步骤 2 AG 侧查询 HFP profile 连接状态，调用 hfp_ag_get_device_connect_state 接口。
- 步骤 3 AG 侧查询 HFP 音频连接状态，调用 hfp_ag_get_sco_connect_state 接口。

----结束

返回值

返回值请参见对应的接口返回值说明。

表13-2 HFP profile 连接状态

序号	定义	实际数值	说明
1	PROFILE_STATE_CONNECTING	1	正在连接。
2	PROFILE_STATE_CONNECTED	2	已连接。
3	PROFILE_STATE_DISCONNECTING	3	正在断开。
4	PROFILE_STATE_DISCONNECTED	4	已断开。

其他函数返回值如表 13-3 所示。

表13-3 其他函数返回值说明

序号	定义	实际数值	说明
1	ERRCODE_BT_SUCCESS	0	执行成功。
2	ERRCODE_BT_FAIL	0x80006000	执行失败。
3	ERRCODE_BT_NOT_READY	0x80006001	执行状态未就绪。
4	ERRCODE_BT_MALLOC_FAIL	0x80006002	内存不足。
5	ERRCODE_BT_MEMCPY_FAIL	0x80006003	内存拷贝错误。
6	ERRCODE_BT_BUSY	0x80006004	繁忙无法响应。
7	ERRCODE_BT_DONE	0x80006005	执行完成。
8	ERRCODE_BT_UNSUPPORTED	0x80006006	不支持。
9	ERRCODE_BT_PARAM_ERR	0x80006007	无效参数。
10	ERRCODE_BT_STATE_ERR	0x80006008	状态错误。
11	ERRCODE_BT_UNHANDLED	0x80006009	未处理错误。
12	ERRCODE_BT_AUTH_FAIL	0x8000600A	鉴权失败。

序号	定义	实际数值	说明
13	ERRCODE_BT_RMT_DEV_DOWN	0x8000600B	远端设备关闭。
14	ERRCODE_BT_AUTH_REJECTED	0x8000600C	鉴权被拒。
15	ERRCODE_BT_MAX	0x800067FF	error code 枚举最大值。

表13-4 HFP 音频连接状态

序号	定义	实际数值	说明
1	HFP_SCO_STATE_CONNECTING	0x01	SCO 正在连接。
2	HFP_SCO_STATE_CONNECTED	0x02	SCO 已连接。
3	HFP_SCO_STATE_DISCONNECTING	0x03	SCO 正在断连。
4	HFP_SCO_STATE_DISCONNECTED	0x04	SCO 已断连。
5	HFP_SCO_STATE_CONNECT_FAILURE	0x05	SCO 连接失败。

表13-5 HFP 的音量调整类型

序号	定义	实际数值	说明
1	HFP_VOLUME_MIC	0x00	调整 microphone 增益。
2	HFP_VOLUME_SPEAKER	0x01	调整 speaker 增益。

表13-6 codec 编码信息

序号	定义	实际数值	说明
1	HFP_AG_CODEC_ID_CVSD	0x01	CVSD 编码格式。
2	HFP_AG_CODEC_ID_MSBC	0x02	MSBC 编码格

序号	定义	实际数值	说明
			式。

表13-7 电话呼叫状态类型

序号	定义	实际数值	说明
1	HFP_AG_CALL_STATE_ACTIVE	0	正在通话。
2	HFP_AG_CALL_STATE_HELD	1	呼叫保持通话。
3	HFP_AG_CALL_STATE_DIALING	2	正在拨号。
4	HFP_AG_CALL_STATE_ALERTING	3	正在响铃。
5	HFP_AG_CALL_STATE_INCOMING	4	呼叫接入。
6	HFP_AG_CALL_STATE_WAITING	5	呼叫等待。

表13-8 服务指示器的值

序号	定义	实际数值	说明
1	HFP_AG_SERVICE_UNAVAILABLE	0	服务不可用。
2	HFP_AG_SERVICE_AVAILABLE	1	服务可用。

表13-9 呼叫指示器的值

序号	定义	实际数值	说明
1	HFP_AG_CALL_NO_ACTIVE	0	无活跃通话。
2	HFP_AG_CALL_ACTIVE	1	正在通话

表13-10 呼叫建立指示器的值

序号	定义	实际数值	说明
----	----	------	----

序号	定义	实际数值	说明
1	HFP_AG_CALLSETUP_NO	0	无呼叫建立。
2	HFP_AG_CALLSETUP_INCOMING	1	来电中。
3	HFP_AG_CALLSETUP_OUTGOING	2	呼出中。
4	HFP_AG_CALLSETUP_OUTALERT	3	响铃中。

表13-11 呼叫保持指示器的值

序号	定义	实际数值	说明
1	HFP_AG_CALLHELD_NO	0	无呼叫保持。
2	HFP_AG_CALLHELD_ACTIVE_HOLD	1	呼叫保持中。
3	HFP_AG_CALLHELD_ONHOLD	2	呼出保持。

表13-12 漫游指示器的值

序号	定义	实际数值	说明
1	HFP_AG_ROAM_NO_ACTIVE	0	无漫游。
2	HFP_AG_ROAM_ACTIVE	1	漫游中。

表13-13 网络事件类型

序号	定义	实际数值	说明
1	HFP_AG_NETWORK_EVENT_RMT_IS_BUSY	0x01	远端忙事件。
2	HFP_AG_NETWORK_EVENT_ALERTING_RM	0x02	响铃远端事件。
3	HFP_AG_NETWORK_EVENT_INCOMING_CALL	0x03	来电事件。
4	HFP_AG_NETWORK_EVENT_RMT_ANSWER_CALL	0x04	远端接听事件。

序号	定义	实际数值	说明
5	HFP_AG_NETWORK_EVENT_SVC_UNAVAILABLE	0x05	服务不可用事件。
6	HFP_AG_NETWORK_EVENT_SVC_AVAILABLE	0x06	服务可用事件。
7	HFP_AG_NETWORK_EVENT_SIGNAL_STRENGTH	0x07	信号强度事件。
8	HFP_AG_NETWORK_EVENT_ROAMING_RESET	0x08	漫游重置事件。
9	HFP_AG_NETWORK_EVENT_ROAMING_ACTIVE	0x09	漫游活跃事件。

表13-14 HFP 通话类型

序号	定义	实际数值	说明
1	HFP_AG_TYPE_ALL_CALLS	0x01	所有存在的通话。
2	HFP_AG_TYPE_INCOMING_CALL	0x02	来电。
3	HFP_AG_TYPE_HELDINCOMING_CALL	0x03	保持状态的来电。
4	HFP_AG_TYPE_OUTGOING_CALL	0x04	呼叫中的通话。
5	HFP_AG_TYPE_ONGOING_CALL	0x05	进行中的通话。

表13-15 AG 挂断类型

序号	定义	实际数值	说明
1	HFP_AG_CANCELED_ALLCALL	0x01	AG 释放所有通话或者 GSM 服务不可用。
2	HFP_AG_CANCELED_CALLSETUP	0x02	AG 或者 GSM 释放来电中的通话或者呼叫中的通话。
3	HFP_AG_CANCELED_LASTCALL	0x03	AG 或者 GSM 释放最后一次通话。

序号	定义	实际数值	说明
4	HFP_AG_CANCELED_CALLHELD	0x04	GSM 释放保持中的通话。

表13-16 扩展的音频网关错误码

序号	定义	实际数值	说明
1	HFP_CMEERR_AGFAILURE	0	AG 失败。
2	HFP_CMEERR_NOCONN2PHONE	1	没有连接到手机。
3	HFP_CMEERR_OPERATION_NOTALLOWED	3	操作不允许。
4	HFP_CMEERR_OPERATION_NOTSUPPORTED	4	操作不支持。
5	HFP_CMEERR_PHSIMPIN_REQUIRED	5	需要 PH-SIM 卡 PIN 码。
6	HFP_CMEERR_SIMNOT_INSERTED	10	没有插入 SIM 卡。
7	HFP_CMEERR_SIMPIN_REQUIRED	11	需要 SIM 卡 PIN 码。
8	HFP_CMEERR_SIMPUK_REQUIRED	12	需要 SIM 卡 PUK 码。
9	HFP_CMEERR_SIM_FAILURE	13	SIM 卡失败。
10	HFP_CMEERR_SIM_BUSY	14	SIM 卡忙。
11	HFP_CMEERR_INCORRECT_PASSWORD	16	不正确的密码。
12	HFP_CMEERR_SIMPIN2_REQUIRED	17	需要 SIM 卡 PIN2 码。
13	HFP_CMEERR_SIMPUK2_REQUIRED	18	需要 SIM 卡 PUK2 码。
14	HFP_CMEERR_MEMORY_FULL	20	内存满。

序号	定义	实际数值	说明
15	HFP_CMEERR_INVALID_INDEX	21	无效索引。
16	HFP_CMEERR_MEMORY_FAILURE	23	内存申请失败。
17	HFP_CMEERR_TEXTSTRING_TOOLONG	24	文本字符串过长。
18	HFP_CMEERR_INVALID_CHAR_INTEXTSTRING	25	文本字符串中的无效字符。
19	HFP_CMEERR_DIAL_STRING_TOOLONG	26	拨号过长。
20	HFP_CMEERR_INVALID_CHAR_INDIALSTRING	27	拨号中的无效字符。
21	HFP_CMEERR_NETWORK_NOSERVICE	30	没有网络服务。
22	HFP_CMEERR_NETWORK_TIMEOUT	31	网络超时。
23	HFP_CMEERR_EMERGENCYCALL_ONLY	32	网络不允许，仅支持紧急呼叫。

表13-17 是否最后一个

序号	定义	实际数值	说明
1	HFP_AG_LAST_NO	0	否
2	HFP_AG_LAST_YES	1	是

表13-18 CLCC 模式

序号	定义	实际数值	说明
1	HFP_AG_CLCC_VOICE	0	语音
2	HFP_AG_CLCC_DATA	1	数据
3	HFP_AG_CLCC_FAX	2	传真

表13-19 是否多方通话

序号	定义	实际数值	说明
1	HFP_AG_CLCC_NOT_MULTI	0	否
2	HFP_AG_CLCC_MULTI	1	是

表13-20 通话方向

序号	定义	实际数值	说明
1	HFP_AG_CLCC_OUTGOING	0	呼出
2	HFP_AG_CLCC_INCOMING	1	来电

表13-21 号码信息服务值

序号	定义	实际数值	说明
1	HFP_AG_CNUM_ASYNC	0	异步
2	HFP_AG_CNUM_SYNC	1	同步
3	HFP_AG_CNUM_PAD	2	PAD
4	HFP_AG_CNUM_PACKET	3	包
5	HFP_AG_CNUM_VOICE	4	语音
6	HFP_AG_CNUM_FAX	5	传真

表13-22 运营商信息格式

序号	定义	实际数值	说明
1	HFP_AG_COPS_LONG_ALPHANUM	0	长格式的字符串数值，最大长度为16Byte。
2	HFP_AG_COPS_SHORT_ALPHANUM	1	短格式的字符串数值，最大长度为

序号	定义	实际数值	说明
			8Byte。
3	HFP_AG_COPS_NUM	2	数值格式。

表13-23 运营商信息模式

序号	定义	实际数值	说明
1	HFP_AG_COPS_AUTO	0	自动。
2	HFP_AG_COPS_MANU	1	手动。
3	HFP_AG_COPS_OFF	2	断开网络。
4	HFP_AG_COPS_DISABLE	3	仅在读命令中配置。
5	HFP_AG_COPS_MANU_AUTO	4	自动/手动自动切换。

13.3 开发实例

以下基于 SDK 包 “include/middleware/services/bts/br/bts_hfp_ag.h” 实现 HFP-AG 服务来电事件通知 HF 实例。具体代码可以参照 SDK 包 “protocol/bt/host/bt/bts/bt_sample/br_profile/sample_hfp_ag_service.c” 中的代码实现。

步骤 1 使能蓝牙协议栈（必选）。

```
errcode_t ret = enable_bt_stack();
return ret;
```

步骤 2 实现 HFP-AG 回调函数。

按照 hfp_ag_answer_call_req_callback 回调函数为例：

```
static void sample_hfp_ag_answer_call_req_callback(const bd_addr_t *bd_addr, hfp_ag_type_t
type)
{
```

```
// 上层业务处理收到的HF端接听请求  
}
```

步骤 3 注册 HFP-AG 回调函数。

```
hfp_ag_callbacks_t cb = {  
    .answer_call_req_cb = sample_hfp_ag_answer_call_req_callback,  
    // 其他回调函数按需注册  
};  
errcode_t ret = hfp_ag_register_callbacks(&cb);  
return ret;
```

步骤 4 与 HF 设备进行 HFP profile 连接。

```
errcode_t ret = hfp_ag_connect(bd_addr);  
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

步骤 5 来电事件通知 HF。

```
char * number_str = "10086";  
hfp_phone_info_t phone_info = { 0 };  
phone_info.type = 128;  
phone_info.service = 4;  
phone_info.num_len = 5;  
memcpy_s(phone_info.number, HFP_AG_PHONENUM_MAX_DIGITS, (uint8_t *)number_str,  
strlen(number_str));  
errcode_t ret = hfp_ag_send_network_incoming_event(bd_addr, &phone_info);  
return (ret == ERRCODE_BT_SUCCESS) ? true : false;
```

----结束

14 GATT 服务特性样例

14.1 概述

14.2 ANCS 客户端

14.3 AMS 客户端

14.1 概述

该章节说明 BLE GATT 服务特性样例，目前提供以下服务样例供参考使用：

- ANCS 客户端
- AMS 客户端

14.2 ANCS 客户端

📖 说明

该服务特性源码位置：

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/ble_ancs_client.c

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/include/ble_ancs_client.h

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/ble_gatts_callbacks.c

AT 指令源码位置：

protocol/bt/host/bt/bts/bt_sample/ble_profile/ble_at_sample/src/sample_ble_ancs.c

protocol/bt/host/bt/bts/bt_sample/ble_profile/ble_at_sample/include/sample_ble_ancs.h

使用场景

与苹果手机 BLE 连接时，实现接收手机的通知消息、获取消息详情、处理消息功能。

官网协议链接

<https://developer.apple.com/library/archive/documentation/CoreBluetooth/Reference/AppleNotificationCenterServiceSpecification/Introduction/Introduction.html>

该服务特性提供功能

1. 发现对端 ANCS 服务，特征及描述符。
2. 订阅通知源特征通知。
3. 订阅数据源特征通知。
4. 发送获取通知属性命令。
5. 发送获取应用属性命令。
6. 发送执行通知行为命令。
7. 解析对端上报的通知消息和属性响应信息，并上报给上层应用。

适配提示

1. 请检查是否完成 GATT Client 注册和 GATT client 回调注册，如果没有，请先调用以下接口处理：
 - a. GATT Client 注册：gattc_register_client，请参照“4 GATT client 接口”章节的表 4-1。
 - b. GATT client 回调注册：gattc_register_callbacks，请参照“4 GATT client 接口”章节的表 4-1。
2. 请使用【发现所有服务】与手机 BLE 建立服务发现，该特性提供的服务建立接口仅供单点测试用。
3. 请根据实际应用场景，控制并发和消息缓存。
4. 目前客户端只支持解析分包数据的第一包，连接 BLE 后，客户端需要主动协商 MTU 为最大 517。

且以下场景，需要限制通知属性长度，以规避对端响应数据超过 MTU 大小而分包发送的情况。

获取通知属性（标题、子标题、消息内容）需要加上最大长度限制，对端响应数据时会根据该长度限制响应数据长度。

接口说明

表14-1 ANCS 客户端接口说明

接口名称	接口说明
ble_ancs_client_register_callbacks	注册 ANCS 客户端回调函数，处理手机推送的通知消息。
ble_ancs_client_setup_service	建立服务，该接口仅限单点测试用，实际商用时请使用 GATT client 接口【gattc_discovery_service】发现所有服务。
ble_ancs_client_discovery_service_cbk	处理发现服务上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现服务回调函数中调用。
ble_ancs_client_discovery_character_cbk	处理发现特征上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现特征回调函数中调用。
ble_ancs_client_discovery_descriptor_cbk	处理发现描述符上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现描述符回调函数中调用。
ble_ancs_client_notify_cbk	处理通知上报回调，在 GATT client 接口【gattc_register_callbacks】注册的通知回调函数中调用。
ble_ancs_client_write_cbk	处理写请求响应上报回调，在 GATT client 接口【gattc_register_callbacks】注册的写响应回调函数中调用。
ble_ancs_client_subscribe_notification_source	订阅通知源特征通知。
ble_ancs_client_subscribe_data_source	订阅数据源特征通知。
ble_ancs_client_send_perform_notify_action_cmd	处理通知消息。
ble_ancs_client_send_get_notify_att_cmd	获取通知消息属性。
ble_ancs_client_send_get_app_name_cmd	获取应用名称。

接口名称	接口说明
t_app_att_cmd	
ble_ancs_client_clear_remote_by_conn	根据连接 ID 清理客户端存储的对端 ANCS 服务信息，断开连接时，清理本端缓存的服务信息。
is_ble_ancs_service	判断发现的服务是否是 ANCS 服务，在 GATT client 接口【gattc_register_callbacks】注册的发现服务回调函数中调用。

接口参数枚举值说明

表14-2 事件 ID

枚举名称	枚举值	枚举说明
ANCS_EVENT_ADDED	0	新增通知
ANCS_EVENT_MODIFIED	1	修改通知
ANCS_EVENT_REMOVEVD	2	移除通知
ANCS_EVENT_RESERVED	预留	预留

表14-3 事件标记

枚举名称	枚举值	枚举说明
ANCS_EVENT_FLAG_SILENT	(1<< 0)	安静
ANCS_EVENT_FLAG_IMPORTANT	(1<< 1)	重要
ANCS_EVENT_FLAG_PREEXISTING	(1<< 2)	先存的
ANCS_EVENT_FLAG_POSITIVE_ACTION	(1<< 3)	积极活动
ANCS_EVENT_FLAG_NEGATIVE_ACTION	(1<< 4)	消极活动
ANCS_EVENT_RESERVED	预留	预留

表14-4 命令 ID

枚举名称	枚举值	枚举说明
ANCS_CMD_GET_NOTIFY_ATT	0	获取通知属性
ANCS_CMD_GET_APP_ATT	1	获取应用属性
ANCS_CMD_PERFORM_NOTIFY_ACTION	2	处理通知
ANCS_CMD_RESERVED	预留	预留

表14-5 分类 ID

枚举名称	枚举值	枚举说明
ANCS_CATEGORY_OTHER	0	其他
ANCS_CATEGORY_INCOMING_CALL	1	来电
ANCS_CATEGORY_MISSED_CALL	2	未接来电
ANCS_CATEGORY_VOICE_MAIL	3	语音消息
ANCS_CATEGORY_SOCIAL	4	社交
ANCS_CATEGORY_SCHEDULE	5	计划
ANCS_CATEGORY_EMAIL	6	邮件
ANCS_CATEGORY_NEWS	7	新闻
ANCS_CATEGORY_HEALTH_FITNESS	8	运动健康
ANCS_CATEGORY_BUSINESS_FINANCE	9	商业金融
ANCS_CATEGORY_LOCATION	10	位置
ANCS_CATEGORY_ENTERTAINMENT	11	娱乐
ANCS_CATEGORY_RESERVED	预留	预留

表14-6 通知属性 ID

枚举名称	枚举值	枚举说明
ANCS_NOTIFY_ATT_APP_ID	0	应用 ID
ANCS_NOTIFY_ATT_TITLE	1	标题
ANCS_NOTIFY_ATT_SUBTITLE	2	子标题
ANCS_NOTIFY_ATT_MESSAGE	3	消息内容
ANCS_NOTIFY_ATT_MSG_SIZE	4	消息大小
ANCS_NOTIFY_ATT_DATE	5	日期
ANCS_NOTIFY_ATT_POSITIVE_LABEL	6	积极行为标签
ANCS_NOTIFY_ATT_NEGATIVE_LABEL	7	消极行为标签
ANCS_NOTIFY_ATT_RESERVED	预留	预留

表14-7 应用属性 ID

枚举名称	枚举值	枚举说明
ANCS_APP_ATT_DISPLAY_NAME	0	应用显示名称
ANCS_APP_ATT_RESERVED	预留	预留

表14-8 活动 ID

枚举名称	枚举值	枚举说明
ANCS_ACTION_POSITIVE	0	积极活动
ANCS_ACTION_NEGATIVE	1	消极活动
ANCS_ACTION_RESERVED	预留	预留

表14-9 订阅类型

枚举名称	枚举值	枚举说明
ANCS_CLIENT_SUBSCRIBE_NOTIFY	0	订阅通知
ANCS_CLIENT_SUBSCRIBE_DATA	1	订阅数据

表14-10 ANCS 客户端写特征响应错误码

枚举名称	枚举值	枚举说明
ANCS_ERROR_CODE_UNKNOW_CMD	0xA0	未知命令
ANCS_ERROR_CODE_INVALID_CMD	0xA1	无效命令
ANCS_ERROR_CODE_INVALID_PARAM	0xA2	无效参数
ANCS_ERROR_CODE_ACTION_FAILED	0xA3	行为没有被执行

表14-11 通知开关

枚举名称	枚举值	枚举说明
ANCS_CLIENT_NOTIFY_DISABLE	0	关闭通知
ANCS_CLIENT_NOTIFY_ENABLE	1	开启通知

14.3 AMS 客户端

说明

该服务特性源码位置：

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/ble_ams_client.c

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/include/ble_ams_client.h

protocol/bt/host/bt/bts/bt_sample/ble_profile/profiles/src/ble_gatts_callbacks.c

AT 指令源码位置：

protocol/bt/host/bt/bts/bt_sample/ble_profile/ble_at_sample/src/sample_ble_ams.c

protocol/bt/host/bt/bts/bt_sample/ble_profile/ble_at_sample/include/sample_ble_ams.h

使用场景

与苹果手机 BLE 连接时，实现控制手机媒体应用，例如播放、暂停、上下一首、音量调节、收藏/取消收藏，手表也可以获取播放器信息、曲目信息、播放列表信息等显示在手表上。

官网协议链接

https://developer.apple.com/library/archive/documentation/CoreBluetooth/Reference/AppleMediaService_Reference/Introduction/Introduction.html

该服务特性提供功能

- 发现对端 AMS 服务，特征及描述符。
- 订阅远端命令特征通知。
- 订阅实体更新特征通知。
- 发送远端命令。
- 注册感兴趣的实体属性。
- 请求对端加载完整实体属性值。
- 读取对端完整实体属性值。
- 解析对端上报的远端命令信息和实体更新信息，并上报给上层应用。

适配提示

1. 请检查是否完成 GATT Client 注册和 GATT client 回调注册，如果没有，请先调用以下接口处理：
 - a. GATT Client 注册：gattc_register_client，请参照 GATT client 接口章节的表 4-1。
 - b. GATT client 回调注册：gattc_register_callbacks，请参照 GATT client 接口章节的表 4-1。
2. 请使用【发现所有服务】与手机 BLE 建立服务发现，该特性提供的服务建立接口仅供单点测试用。
3. 请根据实际应用场景，控制并发和消息缓存。
4. 连接 BLE 后，建议客户端主动协商 MTU 为最大 517。

接口说明

表14-12 AMS 客户端接口说明

接口名称	接口说明
ble_ams_client_register_callbacks	注册 AMS 客户端回调函数，回调函数中处理手机推送的远端命令和实体属性信息。
ble_ams_client_setup_service	建立服务，该接口仅限单点测试用，实际商用时请使用 GATT client 接口【gattc_discovery_service】发现所有服务。
ble_ams_client_discovery_service_cbk	处理发现服务上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现服务回调函数中调用。
ble_ams_client_discovery_character_cbk	处理发现特征上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现特征回调函数中调用。
ble_ams_client_discovery_descriptor_cbk	处理发现描述符上报回调，在 GATT client 接口【gattc_register_callbacks】注册的发现描述符回调函数中调用。
ble_ams_client_notify_cbk	处理通知上报回调，在 GATT client 接口【gattc_register_callbacks】注册的通知回调函数中调用。
ble_ams_client_write_cbk	处理写请求响应上报回调，在 GATT client 接口【gattc_register_callbacks】注册的写响应回调函数中调用。
ble_ams_client_read_cbk	处理读请求响应上报回调，在 GATT client 接口【gattc_register_callbacks】注册的读响应回调函数中调用。
ble_ams_client_subscribe_remote_cmd	订阅远端命令特征通知。
ble_ams_client_subscribe_entity_update	订阅实体更新特征通知。
ble_ams_client_send_remote_cmd	发送远端命令。

接口名称	接口说明
ble_ams_client_register_interested_entity_attribute	注册感兴趣的实体属性信息。
ble_ams_client_retrieve_full_entity_attribute	请求加载完整的实体属性信息。
ble_ams_client_read_full_entity_attribute	读取完整的实体属性信息。
ble_ams_client_clear_remote_by_conn	根据连接 ID 清理客户端存储的对端 AMS 服务信息，断开连接时，清理本端缓存的服务信息。
is_ble_ams_service	判断发现的服务是不是 AMS 服务，在 GATT client 接口【gattc_register_callbacks】注册的发现服务回调函数中调用。

接口参数枚举值说明

表14-13 远端命令 ID

枚举名称	枚举值	枚举说明
AMS_RMT_CMD_ID_PLAY	0	播放
AMS_RMT_CMD_ID_PAUSE	1	暂停
AMS_RMT_CMD_ID_TOGGLE_PLAY_PAUSE	2	切换播放/暂停
AMS_RMT_CMD_ID_NEXT_TRACK	3	下一首
AMS_RMT_CMD_ID_PREVIOUS_TRACK	4	上一首
AMS_RMT_CMD_ID_VOLUME_UP	5	调高音量
AMS_RMT_CMD_ID_VOLUME_DOWN	6	调低音量
AMS_RMT_CMD_ID_ADVANCE_REPEAT_MODE	7	高级重复模式
AMS_RMT_CMD_ID_ADVANCE_SHUFFLE_MODE	8	高级随机模式
AMS_RMT_CMD_ID_SKIP_FORWARD	9	跳过前一个
AMS_RMT_CMD_ID_SKIP_BACKWARD	10	跳过后一个

枚举名称	枚举值	枚举说明
AMS_RMT_CMD_ID_LIKE_TRACK	11	喜欢
AMS_RMT_CMD_ID_DISLIKE_TRACK	12	取消喜欢
AMS_RMT_CMD_ID_BOOKMARK_TRACK	13	标记喜欢或取消喜欢
AMS_RMT_CMD_ID_RESERVED	预留	预留

表14-14 实体 ID

枚举名称	枚举值	枚举说明
AMS_ENTITY_ID_PLAYER	0	播放器
AMS_ENTITY_ID_QUEUE	1	播放列表
AMS_ENTITY_ID_TRACK	2	曲目
AMS_ENTITY_ID_RESERVED	预留	预留

表14-15 播放器属性 ID

枚举名称	枚举值	枚举说明
AMS_PALYER_ATT_ID_NAME	0	播放器名称
AMS_PALYER_ATT_ID_PLAYBACK_INFO	1	播放信息
AMS_PALYER_ATT_ID_VOLUME	2	音量, 0~1 的浮点数
AMS_PALYER_ATT_ID_RESERVED	预留	预留

表14-16 队列属性 ID

枚举名称	枚举值	枚举说明
AMS_QUEUE_ATT_ID_INDEX	0	序号, 从 0 开始
AMS_QUEUE_ATT_ID_COUNT	1	列表个数

枚举名称	枚举值	枚举说明
AMS_QUEUE_ATT_ID_SHUFFLE_MODE	2	随机模式
AMS_QUEUE_ATT_ID_REPEAT_MODE	3	重复模式
AMS_QUEUE_ATT_ID_RESERVED	预留	预留

表14-17 曲目属性 ID

枚举名称	枚举值	枚举说明
AMS_TRACK_ATT_ID_ARTIST	0	艺术家
AMS_TRACK_ATT_ID_ALBUM	1	专辑
AMS_TRACK_ATT_ID_TITLE	2	标题
AMS_TRACK_ATT_ID_DURATION	3	音乐时长, 秒级, 浮点数
AMS_TRACK_ATT_ID_RESERVED	预留	预留

表14-18 随机模式

枚举名称	枚举值	枚举说明
AMS_SHUFFLE_MODE_OFF	0	关闭
AMS_SHUFFLE_MODE_ONE	1	随机模式 1
AMS_SHUFFLE_MODE_ALL	2	随机模式 2
AMS_SHUFFLE_MODE_RESERVED	预留	预留

表14-19 重复模式

枚举名称	枚举值	枚举说明
AMS_REPEAT_MODE_OFF	0	关闭

枚举名称	枚举值	枚举说明
AMS_REPEAT_MODE_ONE	1	重复模式 1
AMS_REPEAT_MODE_ALL	2	重复模式 2
AMS_REPEAT_MODE_RESERVED	预留	预留

表14-20 AMS 客户端写和读特征响应错误码

枚举名称	枚举值	枚举说明
AMS_ERROR_CODE_INVALID_STATE	0xA0	无效状态，客户端未发起服务建立
AMS_ERROR_CODE_INVALID_CMD	0xA1	无效命令
AMS_ERROR_CODE_ABSENT_ATTRIBUTE	0xA2	属性为空

表14-21 实体更新标记

枚举名称	枚举值	枚举说明
AMS_ENTITY_UPDATE_FLAG_TRUNCATED	(1<< 0)	销毁
AMS_ENTITY_UPDATE_FLAG_RESERVED	预留	预留

表14-22 订阅类型

枚举名称	枚举值	枚举说明
AMS_CLIENT_SUBSCRIBE_REMOTE_CMD	0	订阅远端命令
AMS_CLIENT_SUBSCRIBE_ENTITY_UPDATE	1	订阅实体更新

表14-23 写特征类型

枚举名称	枚举值	枚举说明
------	-----	------

枚举名称	枚举值	枚举说明
AMS_CLIENT_WRITE_REMOTE_CMD	0	写远端命令
AMS_CLIENT_WRITE_ENTITY_UPDATE	1	写实体更新
AMS_CLIENT_WRITE_ENTITY_ATTRIBUTE	2	写实体属性

表14-24 通知开关

枚举名称	枚举值	枚举说明
AMS_CLIENT_NOTIFY_DISABLE	0	关闭通知
AMS_CLIENT_NOTIFY_ENABLE	1	开启通知

15

常见问题定位指南

说明

AT 指令源码位置:

指令定义函数: application/wstp/at_service/bt/bt_manager_cmd_manager.c

指令处理函数: application/wstp/at_service/bt/bt_manager_at_process.c

“AT^BTTESTCMD”开头的指令处理函数: protocol/bt/host/bt/bts/bt_sample/bt_srv_shell.c

15.1 蓝牙连接相关问题

15.2 蓝牙广播相关问题

15.3 蓝牙配对相关问题

15.4 维测指令

15.5 BLE GATT 相关问题

15.1 蓝牙连接相关问题

蓝牙连接相关问题，请检查以下要点：

1. 双模设备蓝牙初始化时，参照 AT 指令 “AT^BTENABLE” 样例代码，执行以下操作：
 - 调用 “gap_register_callbacks” 接口注册 BR GAP 回调。
 - 调用 A2DP、媒体、AVRCP 相关回调接口。
 - 调用 “gap_ble_register_callbacks” 注册 BLE GAP 回调。
 - 调用 “enable_bt_stack” 接口使能 BT，完成初始化。
2. 初始化结果，扫描结果和连接结果，请检查以下回调函数上报信息。

表15-1 初始化和连接相关回调函数

场景	回调函数
初始化状态上报	gap_state_changed_callback
BR 连接状态上报	gap_acl_state_changed_callback
BLE 连接状态上报	gap_ble_connect_state_changed_callback
BR 扫描结果上报	gap_discovery_result_callback
BR 扫描状态上报	gap_discovery_state_changed_callback

3. 扫描结果其他信息可以通过以下接口获取：
- 获取远端设备名称：gap_get_device_name
 - 获取远端设备 RSSI 信息：gap_read_remote_rssi_value
 - 获取远端设备类型：gap_get_device_class

地址参数字节序，请参照表 15-2。

表15-2 地址参数字节序

地址参数字节序	SDK 接口
大端序	bluetooth_set_local_addr
大端序	gap_ble_set_local_addr
小端序	其他需要地址参数的接口

15.2 蓝牙广播相关问题

蓝牙广播相关问题，请检查以下要点：

- 关闭 BR 的广播，参照 AT 指令 “AT+BTINQSCANCLOSE” 样例代码，调用 gap_br_set_bt_scan_mode 接口。
- 打开 BR 的广播，参照 AT 指令 “AT+BTINQSCANOPEN” 样例代码，调用 gap_br_set_bt_scan_mode 接口。

- 调用 bluetooth_set_local_addr 接口后，BR 会重启广播。
- BR 断连之后，如果有配对信息，BR 广播模式会被设置为可连接；如果没有配对信息，BR 广播模式会被设置为可扫描可连接。
- 启动 BLE 广播，请参照以下顺序使用接口：
 - a. 调用 gap_ble_set_adv_param 接口配置广播参数。
 - b. 调用 gap_ble_set_adv_data 接口配置广播数据，该接口为可选。
 - c. 调用 gap_ble_start_adv 接口启动广播。
- BLE 广播数据格式，请参照蓝牙官网协议“BLUETOOTH CORE SPECIFICATION Version 5.3 | Vol 3, Part C page 1357”定义。
- BLE 默认广播数据为设备名称，调用 gap_ble_set_adv_data 接口后，会覆盖默认的广播数据，广播数据需要包含设备名称，否则其他设备可能扫描不到。
- 默认 BLE 连接后，BLE 停止广播，断开连接后，需要重新启动广播。

15.3 蓝牙配对相关问题

蓝牙配对相关问题，请检查以下要点：

- BLE 默认安全参数为可绑定，无 IO 能力，无安全连接，需要修改时，请调用 gap_ble_set_sec_param 接口。
- 当手表端有 BLE HID 服务时，苹果手机才会主动发起配对。
- 配对结果，请检查如表 15-3 所示回调函数上报信息。

表15-3 配对结果回调函数

场景	回调函数
BR 配对结果上报	gap_pair_status_changed_callback
BLE 配对结果上报	gap_ble_paired_complete_callback
BLE 配对密钥信息上报	gap_ble_auth_complete_callback

15.4 维测指令

1. 查询远端设备信息
 - 指令：AT+BTREMOTEINFO?
 - 查询信息：设备名称、设备地址、设备类型、状态、RSSI。
2. 查询 BR 连接信息
 - 指令：AT+BTCONNINFO?
 - 查询信息：设备名称、设备地址、连接状态、链路类型、是否为 master、是否是手机。
3. 查询链路状态
 - 指令：AT+BTLINKTYPE?
 - 查询信息：设备名称、设备地址、链路状态。
4. 查询 BLE 连接信息
 - 指令：AT+BLECONNLIST?
 - 查询信息：设备地址、地址类型、连接 ID、配对状态、连接时隙、连接时延、连接超时时间。

15.5 BLE GATT 相关问题

1. 遇到“btsrv_send_simple_msg: send fail”问题时，请检查以下要点
BLE GATT 回调上报数据频繁时，例如数传场景，回调函数处理逻辑中，不要打印串口日志，不要有耗时的操作。
2. GATT MTU 交换问题，请注意以下要点：
 - GATT MTU 交换需要 client 端发起，如果板侧作 client 端，请调用 gattc_exchange_mtu_req 接口发起 MTU 交换请求。
 - 如果板侧作 server，只能等对端 client 发起 MTU 交换请求，协商结果通过回调函数 gatts_mtu_changed_callback 上报。
 - 默认 MTU 为 23，连接之前不建议修改该值。请在 BLE 连接后，由手表或手机发起 MTU 交换。
 - GATT MTU 范围，最小为 23Byte，最大为 517Byte。